

Quake III DeFRaG Physics

March 19, 2022

Contents

1	Fundamentals	3
1.1	Input	3
1.2	Friction	3
1.3	Acceleration	3
2	CGazHUD	6
2.1	Strafing on flat ground or in air	6
2.2	Strafing on icy surfaces	10
2.3	Strafing on sloped surfaces	10
2.4	CPM air control	12
2.5	Speed gains from air strafing	12
2.6	Speed gains from ground strafing	13
2.7	Swimming and flying	14
3	SnapHUD	19
3.1	Basics	19
3.2	Frame rate impact	19
3.3	Proper maximum ground speed	19
3.4	Speed loss while strafing optimally	20
4	Miscellaneous	21
4.1	Influence of movement keys	21
4.2	Upmove effect on acceleration	23
4.3	Frame rate dependency	24
4.4	Approach to z-axis translations	25
4.5	Step-up	27
4.6	Skimming	27
4.7	Overbounces	28
4.8	Wallbug	28
5	Weapons	29
6	RR ranking system	30
7	TODO	36
A	Definitions	37
B	Code snippets	38
B.1	AngleVectors	38
B.2	PM_Friction	38
B.3	PM_CmdScale	39

C	Derivations	41
C.1	$\delta_{\min}$	41
C.2	δ_{opt}	42
C.3	$\delta_{\max}$	43
C.4	$\tilde{\delta}_{\min}$	45
C.5	$\tilde{\delta}_{\max}$	45
C.6	$\underline{\nu}_{\max}$	46
C.6.1	$\underline{\nu}_{\max}$ alternative	47
C.7	$x_{z,k}$	48
D	Field of view	49
	References	50

1 Fundamentals

[Cho19]

1.1 Input

1.2 Friction

Before any acceleration comes into play, friction is applied first. The amount of friction depends on the player's speed v , the friction factor c ¹ and the frame-time T ². At lower speeds, the stopspeed limit d ³ kicks in and brings the player to a complete stop faster. The current velocity vector before and after friction is applied can then be defined as respectively

$$\mathbf{v} = \begin{pmatrix} v_x \\ v_y \\ v_z \end{pmatrix} = v \hat{\mathbf{v}}, \quad \mathbf{v}_f = \begin{pmatrix} v_{fx} \\ v_{fy} \\ v_{fz} \end{pmatrix} = v_f \hat{\mathbf{v}},$$

with the same direction $\hat{\mathbf{v}}$ and the magnitude relation

$$v_f = (1 - \iota c T) v, \quad (1)$$

assuming $cT \leq 1$ and where

$$\iota = \max \left(1, \min \left(\frac{d}{v}, \frac{1}{cT} \right) \right)^4,$$

or otherwise

$$v_f = \begin{cases} (1 - cT)v, & v \geq d \\ \left(1 - \frac{d}{v}cT\right)v, & dcT < v < d \\ 0, & v \leq dcT, \end{cases}$$

which is derived from Appendix B.2.

This means that, without acceleration, the current velocity either remains unchanged or decreases over time ($0 \leq v_f \leq v$).

1.3 Acceleration

This would not be an interesting game without the ability to increase speed and change direction. So let's define the acceleration vector as

$$\mathbf{a} = \begin{pmatrix} a_x \\ a_y \\ a_z \end{pmatrix} = a \hat{\mathbf{a}},$$

and the resulting new velocity becomes

$$\mathbf{r} = \mathbf{v}_f + \mathbf{a} = \begin{pmatrix} r_x \\ r_y \\ r_z \end{pmatrix} = r \hat{\mathbf{r}}.$$

The relation between all defined vectors is summarized in Figure 1, where the normal of the (horizontal) xy -plane (■) is defined as

$$\mathbf{n} = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}.$$

¹On the ground the friction factor $c = 6$ (pm_friction), but in the air $c = 0$.

²The frame-time $T = 1/125$ fps = 0.008 s.

³The stopspeed limit $d = 100$ ups (pm_stopspeed).

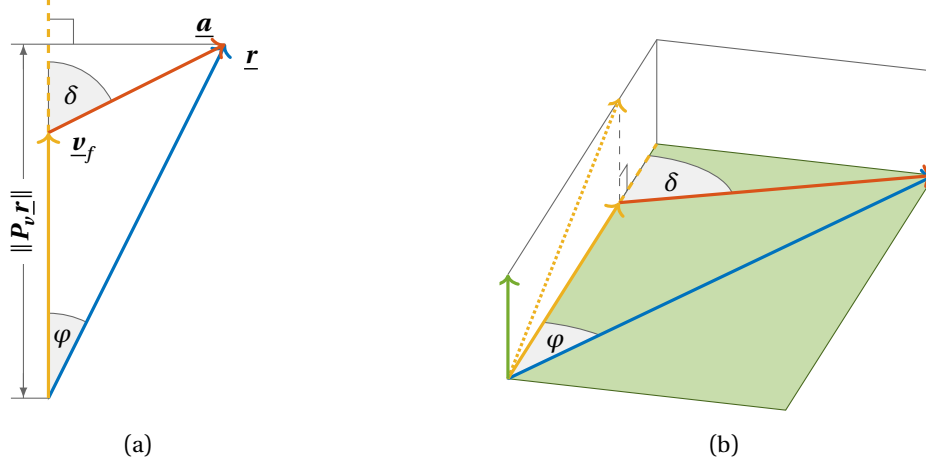


Figure 1: The surface normal $\underline{n}$ ($\rightarrow$), the current velocity $\underline{v}_f$ ($\rightarrow$) and its projection $\underline{v}_f$ ($\rightarrow$), the projected acceleration $\underline{a}$ ($\rightarrow$) and the resulting new velocity $\underline{r}$ ($\rightarrow$). (a) is the top view of (b).

The matrix that projects all vectors orthogonally onto the xy -plane ($\square$) is then

$$\underline{M}_n = \underline{I} - \underline{n}\underline{n}^T = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{pmatrix}$$

Hence, the current velocity vectors projected to the xy -plane become

$$\underline{v} = \underline{M}_n \underline{v} = \begin{pmatrix} v_x \\ v_y \\ 0 \end{pmatrix} = \underline{v} \hat{\underline{v}},$$

$$\underline{v}_f = \underline{M}_n \underline{v}_f = \begin{pmatrix} v_{fx} \\ v_{fy} \\ 0 \end{pmatrix} = \underline{v}_f \hat{\underline{v}}$$

and the projected equivalent of equation (1) is

$$\underline{v}_f = (1 - \iota c T) \underline{v}. \quad (2)$$

Likewise, the projected acceleration vector is

$$\underline{a} = \underline{M}_n \underline{a} = \begin{pmatrix} a_x \\ a_y \\ 0 \end{pmatrix} = \underline{a} \hat{\underline{a}}, \quad (3)$$

with a magnitude

$$\underline{a} = sAT, \quad (4)$$

where A is some dimensionless constant depending on the player state⁵ and s is the wishspeed limit (e.g. $g_speed = 320$ ups on a flat surface under normal conditions, see more details in Section 4.1).

Acceleration magnitude is provided by below code

```
251 accelSpeed = accel*pm1.frameTime*wishSpeed;
```

Listing 1: Code snippet from `code/game/bg_pmove.c`.

In the same way, the projection of the resulting new velocity becomes

$$\underline{r} = \underline{M}_n \underline{r} = \begin{pmatrix} r_x \\ r_y \\ 0 \end{pmatrix} = \underline{r} \hat{\underline{r}}.$$

⁵In the air, $A = 1$ (`pm_airaccelerate`), while on the ground $A = 10$ and $A = 15$ (`pm_accelerate`) for VQ3 and CPM, respectively.

Thus, to project a vector onto the current velocity vector $\underline{v}$ lying on the surface (), it becomes

$$\mathbf{P}_{\underline{v}} = \frac{\underline{v}\underline{v}^T}{\underline{v}^T\underline{v}} = \frac{1}{v_x^2 + v_y^2} \begin{pmatrix} v_x^2 & v_x v_y & 0 \\ v_x v_y & v_y^2 & 0 \\ 0 & 0 & 0 \end{pmatrix},$$

$$\|\mathbf{P}_{\underline{v}}\underline{r}\| = \sqrt{\underline{r}^T \mathbf{P}_{\underline{v}} \underline{r}} = \sqrt{\underline{r}^T \hat{\underline{v}} \hat{\underline{v}}^T \underline{r}} = \underline{r}^T \hat{\underline{v}} = \underline{r} \cos \varphi,$$

where φ represents the angle between the current velocity $\underline{v}$ and the new velocity $\underline{r}$. Likewise, to project a vector onto the acceleration vector $\underline{a}$, we need to multiply with

$$\mathbf{P}_{\underline{a}} = \frac{\underline{a}\underline{a}^T}{\underline{a}^T\underline{a}} = \mathbf{P}_{\underline{a}}^T = \mathbf{P}_{\underline{a}}^k, \quad k \in \mathbb{N}^+,$$

$$\|\mathbf{P}_{\underline{a}}\underline{v}_f\| = \sqrt{\underline{v}_f^T \mathbf{P}_{\underline{a}} \underline{v}_f} = \sqrt{\underline{v}_f^T \hat{\underline{a}} \hat{\underline{a}}^T \underline{v}_f} = \underline{v}_f^T \hat{\underline{a}} = \underline{v}_f \cos \delta,$$

where δ represents the angle between the acceleration $\underline{a}$ and the velocity $\underline{v}$.

2 CGazHUD

[Chr19]

Short for CampingGaz-HUD.

2.1 Strafing on flat ground or in air

When we try to apply an acceleration $\underline{a}$ at some angle δ to the velocity $\underline{v}_f$, we will be allowed to accelerate provided that the component of our current velocity in the proposed direction is smaller than s , also expressed as $\underline{v}_f \cos \delta < s$. The size of the acceleration applied, $\underline{a}$, is a constant⁶, independent of the angle at which it is applied. However, there is one special case which is an exception to this rule. If $\underline{v}_f \cos \delta < s$ but still very close to s , applying the usual constant acceleration $\underline{a}$ for a single frame would be enough to take you over s in that direction. This is something the engine checks for, and if the situation exists, the applied acceleration will be only enough to take you up to s (so the resulting acceleration would be $s - \underline{v}_f \cos \delta$ instead of the full acceleration $\underline{a}$). However, the range of angle δ at which this situation exists is very small ($\delta_{\min} \leq \delta \leq \delta_{\text{opt}}$) as can be seen in Figure 2a–2b.

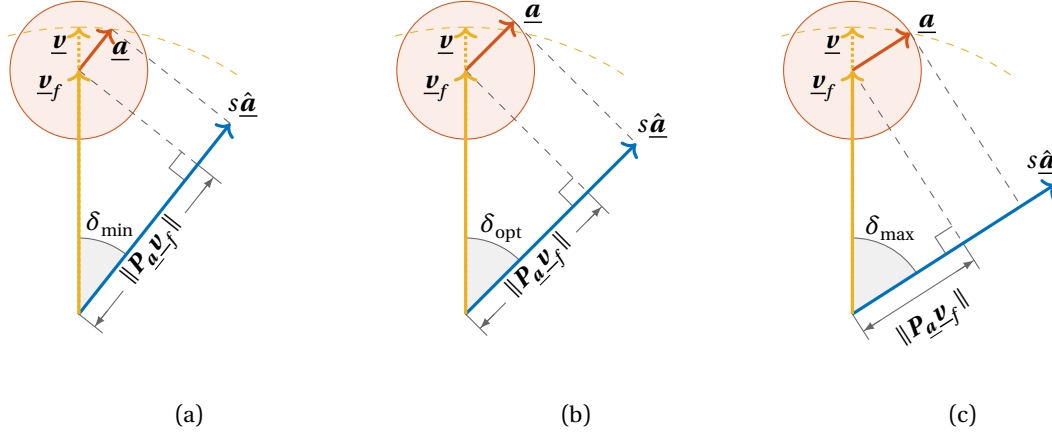


Figure 2: The current velocity $\underline{v}$ (.....) and the velocity after friction $\underline{v}_f$ (—) together with all possible acceleration configurations (□).

Hence, we can summarize that

$$\underline{r} = \begin{cases} \underline{v}_f, & s - \underline{v}_f \cos \delta \leq 0, \\ \underline{v}_f + (s - \underline{v}_f \cos \delta) \hat{\underline{a}}, & 0 < s - \underline{v}_f \cos \delta \leq \underline{a}, \\ \underline{v}_f + \underline{a} \hat{\underline{a}}, & s - \underline{v}_f \cos \delta > \underline{a}. \end{cases} \quad (5)$$

The acceleration causes a change in the total velocity, which is made up of a *change in speed* and a *change in direction*. The direction $\hat{\underline{a}}$ in which the acceleration is applied dictates how much of this acceleration goes into changing the speed, and how much goes into changing the direction. In *strafe-jumping*, it is the *change in speed* we are interested in *maximizing*, in an increasing fashion.

The way to achieve this maximum increase in speed is by applying the acceleration so its direction is as close to that of the original velocity as possible, i.e. so the angle δ is as small as possible. This will mean that more of the acceleration goes into increasing our speed, and less goes into affecting the direction. The limiting condition gives a minimum angle of

$$\delta_{\min} = \arccos \left(\frac{\sqrt{s^2 - \underline{v}^2 + \underline{v}_f^2}}{\underline{v}_f} \right). \quad (6)$$

⁶In the air $\underline{a} = 2.56$, while on the ground $\underline{a} = 25.6$ and $\underline{a} = 38.4$ for VQ3 and CPM, respectively.

However, in order to gain the full acceleration, $\underline{a}$, we need the angle to be a bit larger, so we avoid the special case mentioned above. Therefore the optimal angle is the smallest angle at which we receive the full acceleration, $\underline{a}$, which is given by

$$\delta_{\text{opt}} = \text{acos}\left(\frac{s - \underline{a}}{\underline{v}_f}\right). \quad (7)$$

As you can see, this optimal angle depends on the current velocity $\underline{v}_f$ and increases towards 90° as the velocity goes to infinity. The other limiting condition (i.e. the maximum angle), shown in Figure 3d, is given by

$$\delta_{\text{max}} = \text{acos}\left(\frac{\underline{v}^2 - \underline{v}_f^2 - \underline{a}^2}{2\underline{a}\underline{v}_f}\right). \quad (8)$$

Hence, the angles δ_{min} and δ_{max} represent the boundaries between a speed increase and possible speed decrease. When $\underline{v} = \underline{v}_f$, e.g. in the air⁷, equations (6) and (8) simplify to respectively

$$\delta_{\text{min}} = \text{acos}\left(\frac{s}{\underline{v}}\right), \quad (9)$$

$$\delta_{\text{max}} = \text{acos}\left(-\frac{\underline{a}}{2\underline{v}}\right). \quad (10)$$

Note that, in the air, δ_{max} exceeds 90° and still results in a speed increase. On the ground, the angles δ_{min} , δ_{opt} and δ_{max} will eventually meet when we reach the *maximum ground speed*⁸, due to the friction ($c = 6$), shown in Figure 5 (top left magnification). The mathematical derivation is done in Appendix C.6 and results in

$$\underline{v}_{\text{max}} = s\sqrt{\frac{A(2 - AT)}{c(2 - cT)}}. \quad (11)$$

Knowing this, we find that on a flat surface the maximum ground speed for VQ3 = 409.7180 ups and CPM = 496.5454 ups, at 125 fps. Note that if there is no friction ($c = 0$), there would in principle be no limit on the maximum speed. This would be an example of being on an icy surface, or in the air. However there are inevitably other aspects of the engine that put limits on very high speeds.

⁷In the air there is no friction ($c = 0$).

⁸When we introduce velocity snapping (Section 3.3), this is no longer the maximum ground speed.

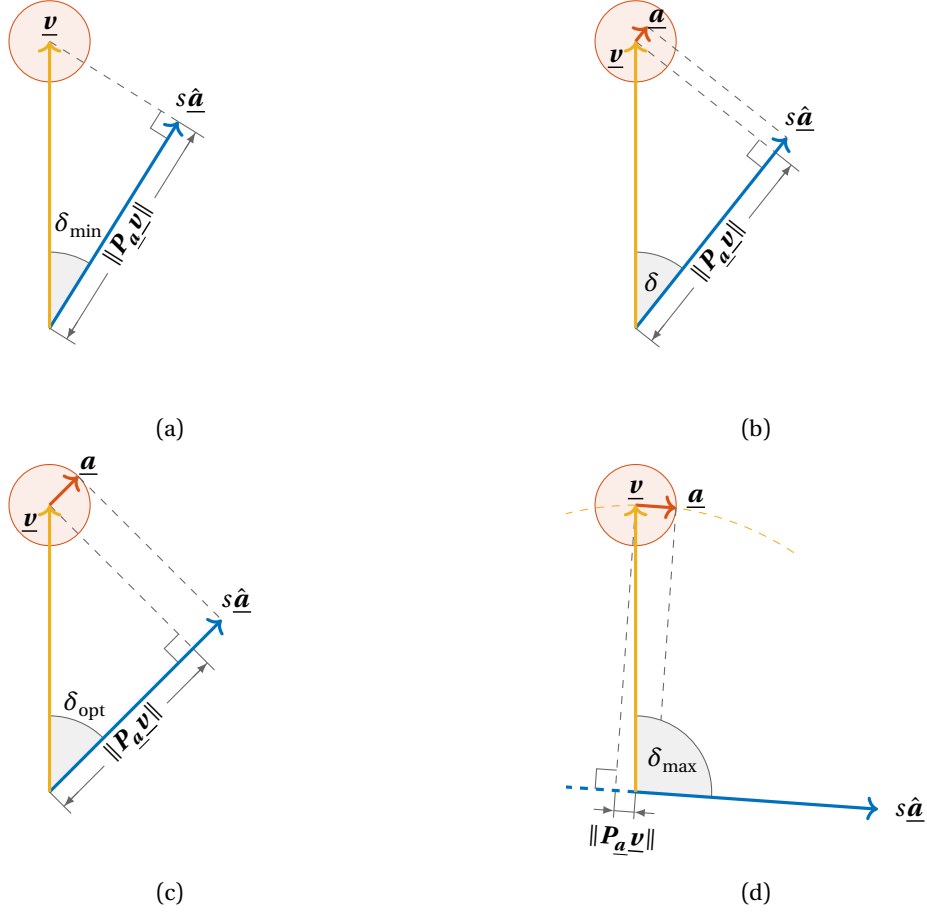


Figure 3: In the air ($\underline{v} = \underline{v}_f$). The current velocity $\underline{v}$ ($\rightarrow$) together with all possible acceleration configurations ($\square$).

At this point, it might be interesting to define the δ -region where you gain speed in the direction of the current velocity $\underline{\hat{v}}$. Similar to equation (6), we derive the point where we first start gaining speed in the direction $\underline{\hat{v}}$. The limiting condition gives a minimum angle of

$$\bar{\delta}_{\min} = \arccos \left(\frac{s + \sqrt{s^2 - 4\underline{v}_f(\underline{v} - \underline{v}_f)}}{2\underline{v}_f} \right). \quad (12)$$

Similar to equation (8), we derive the point where we stop gaining speed in the direction $\underline{\hat{v}}$. The limiting condition gives a maximum angle of

$$\bar{\delta}_{\max} = \arccos \left(\frac{\underline{v} - \underline{v}_f}{\underline{a}} \right). \quad (13)$$

Hence, the angles $\bar{\delta}_{\min}$ and $\bar{\delta}_{\max}$ represent the boundaries between a speed increase and a speed decrease in the direction of the current velocity $\underline{\hat{v}}$. Similar as before, when $\underline{v} = \underline{v}_f$ (e.g. in the air), equations (12) and (13) simplify to

$$\bar{\delta}_{\min} = \arccos \left(\frac{s}{\underline{v}} \right) = (9), \quad (14)$$

$$\bar{\delta}_{\max} = \frac{\pi}{2}. \quad (15)$$

Knowing all these angles, we can create a CGazHUD (i.e. a strafe-jump helper), which is divided in 5 regions (see Figure 4 and 5):

- $\delta \in [0, \delta_{\min})$
The no-acceleration zone.
- $\delta \in [\delta_{\min}, \delta_{\text{opt}})$
The zone where we do not get the full acceleration $\underline{a}$ yet, as described above. Note that it is not desired for a human to stay inside this zone.
- $\delta \in [\delta_{\text{opt}}, \bar{\delta}_{\max})$
The main acceleration zone. Stay for the most part inside this zone, preferably close to the edge between the two green zones to keep the *change in direction* minimal.
- $\delta \in [\bar{\delta}_{\max}, \delta_{\max}]$
The turn zone where we no longer gain speed in the direction of the current velocity $\underline{v}$, however the overall speed increases.
- $\delta \in (\delta_{\max}, \pi]$
The deceleration zone.

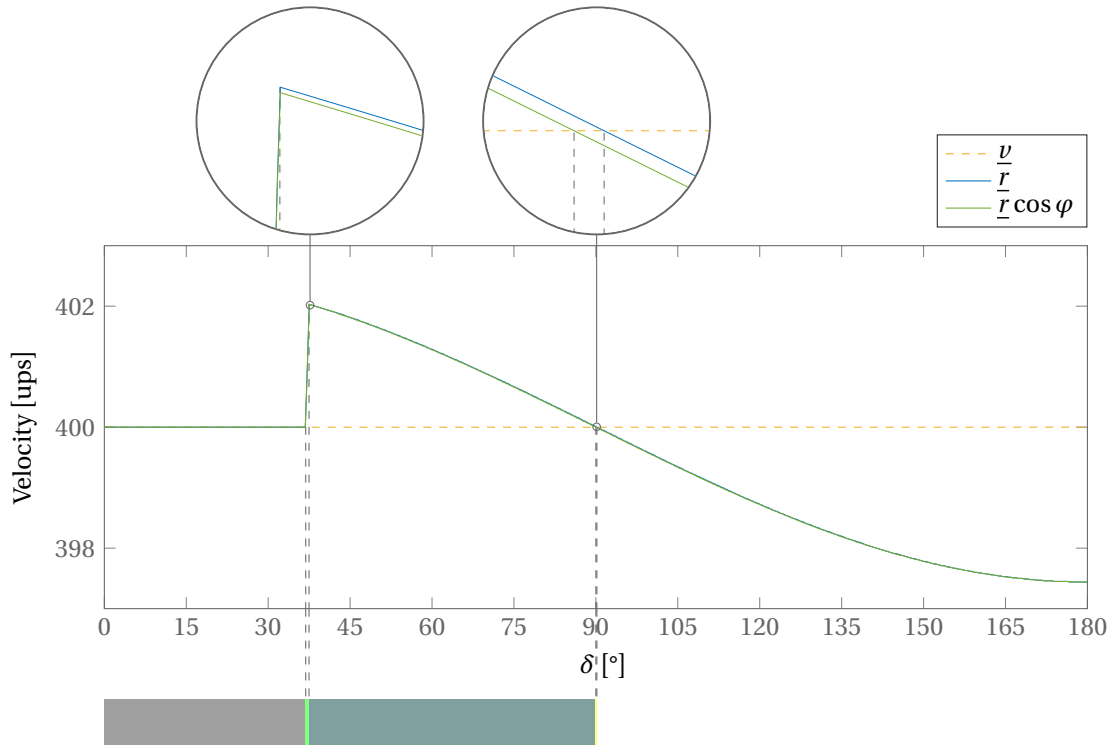


Figure 4: Numerical example in the air with an initial velocity $\mathbf{v} = (400 \ 0 \ 0)^T$. Magnification $\times 30$. CGazHUD boundaries: $0 \leq \delta_{\min}^{(9)} \leq \delta_{\text{opt}}^{(7)} \leq \bar{\delta}_{\max}^{(15)} \leq \delta_{\max}^{(10)} \leq \pi$.

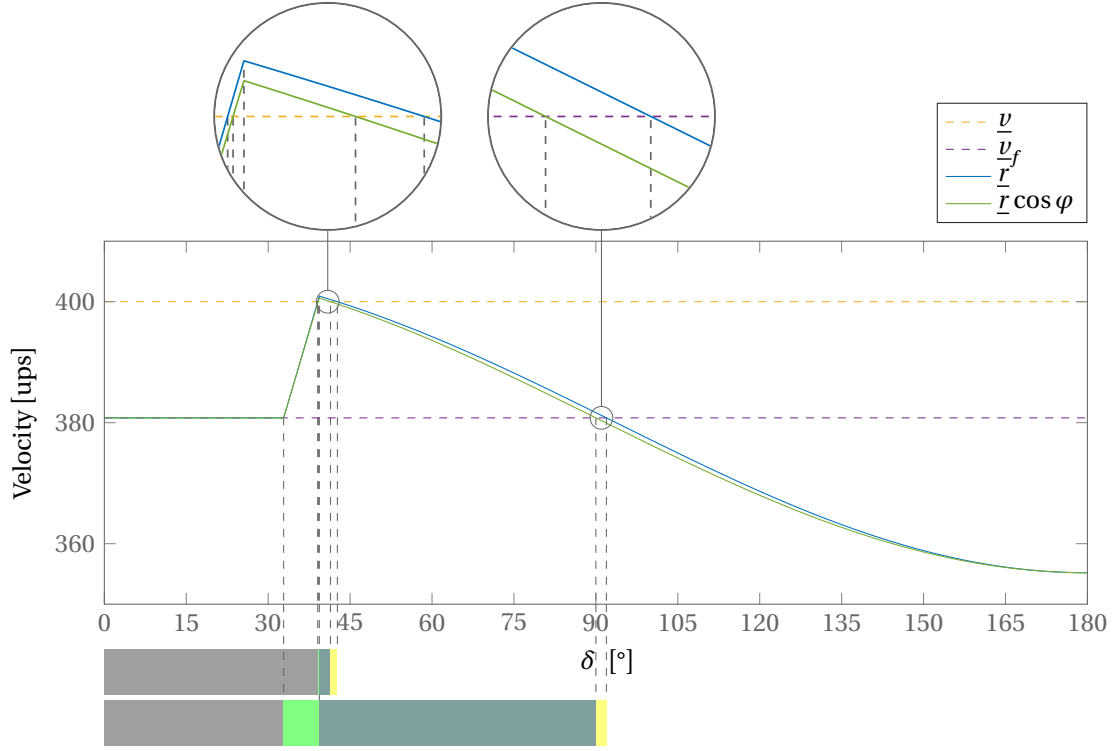


Figure 5: Numerical example on the ground with an initial velocity $\mathbf{v} = (400 \ 0 \ 0)^T$. Magnification $\times 10$. Upper CGazHUD boundaries: $0 \leq \delta_{\min}^{(6)} \leq \delta_{\text{opt}}^{(7)} \leq \delta_{\max}^{(13)} \leq \delta_{\max}^{(8)} \leq \pi$; lower CGazHUD boundaries: $0 \leq \delta_{\min}^{(9)} \leq \delta_{\text{opt}}^{(7)} \leq \delta_{\max}^{(15)} \leq \delta_{\max}^{(10)} \leq \pi$.

2.2 Strafing on icy surfaces

Acceleration $A = 1$ on a icy surface is identical to the acceleration in the air, however, this is not the entire story. Every frame Gravity $g = 800$ (g_gravity).

$$\mathbf{v}_{f_z} - gT$$

TODO: Explain the drifting speed and sticky jump overbounces (graph time for ob with respect to your current velocity, slick top view turn (with(out) jump)).

2.3 Strafing on sloped surfaces

$$\mathbf{n} = \begin{pmatrix} n_1 \\ n_2 \\ n_3 \end{pmatrix}, \quad \mathbf{k} = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix},$$

$$\underline{\mathbf{n}} = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}, \quad \bar{\mathbf{k}} = \begin{pmatrix} -n_1 \\ -n_2 \\ n_3 \end{pmatrix},$$

$$\mathbf{P}_{\perp \mathbf{n} \rightarrow \mathbf{n}} = \mathbf{I} - \mathbf{n} \mathbf{n}^T = \begin{pmatrix} n_2^2 + n_3^2 & -n_1 n_2 & -n_1 n_3 \\ -n_1 n_2 & n_1^2 + n_3^2 & -n_2 n_3 \\ -n_1 n_3 & -n_2 n_3 & n_1^2 + n_2^2 \end{pmatrix},$$

$$\mathbf{P}_{\perp \mathbf{n} \rightarrow \mathbf{k}} = \mathbf{I} - \frac{\mathbf{n} \mathbf{k}^T}{\mathbf{n}^T \mathbf{k}} = \begin{pmatrix} 1 & 0 & -\frac{n_1}{n_3} \\ 0 & 1 & -\frac{n_2}{n_3} \\ 0 & 0 & 0 \end{pmatrix},$$

$$\mathbf{P}_{\perp \mathbf{k} \rightarrow \bar{\mathbf{k}}} = \mathbf{I} - \frac{\mathbf{k} \bar{\mathbf{k}}^T}{\mathbf{k}^T \bar{\mathbf{k}}} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ \frac{n_1}{n_3} & \frac{n_2}{n_3} & 0 \end{pmatrix},$$

$$\begin{aligned}
\text{yaw}_1 &= -\arctan \frac{n_1}{n_2}, \\
\text{yaw}_2 &= -\arctan \frac{n_1}{n_2} + \frac{\pi}{2}, \\
\text{yaw}_3 &= -\arctan \frac{n_1}{n_2} + \pi, \\
\text{yaw}_4 &= -\arctan \frac{n_1}{n_2} - \frac{\pi}{2},
\end{aligned}$$

It is this acceleration, provided by the movement keys, which gives the player a change in velocity, which ultimately changes his position.

$$\underline{\text{fm}} = \begin{pmatrix} \cos(\delta + \text{yaw}_4) \\ \sin(\delta + \text{yaw}_4) \\ 0 \end{pmatrix} = \frac{1}{\sqrt{n_1^2 + n_2^2}} \begin{pmatrix} n_2 \sin \delta - n_1 \cos \delta \\ -n_1 \sin \delta - n_2 \cos \delta \\ 0 \end{pmatrix},$$

One can use the *Rodrigues' rotation formula* to construct the rotation matrix $\mathbf{R}$ that rotates by an angle $\phi = \arccos(\mathbf{k}^T \mathbf{n})$ about the unit vector ($\rightarrow$)

$$\mathbf{u} = \begin{pmatrix} u_1 \\ u_2 \\ u_3 \end{pmatrix} = \frac{1}{\sqrt{n_1^2 + n_2^2}} \begin{pmatrix} -n_2 \\ n_1 \\ 0 \end{pmatrix}. \quad (16)$$

Letting

$$\mathbf{W} = \begin{pmatrix} 0 & -u_3 & u_2 \\ u_3 & 0 & -u_1 \\ -u_2 & u_1 & 0 \end{pmatrix},$$

the Rodrigues' rotation matrix is constructed as

$$\begin{aligned}
\mathbf{R} &= \mathbf{I} + \sin \phi \mathbf{W} + 2 \sin^2 \frac{\phi}{2} \mathbf{W}^2, \\
\mathbf{R} &= \mathbf{I} + \sqrt{n_1^2 + n_2^2} \mathbf{W} + (1 - n_3) \mathbf{W}^2, \\
\mathbf{R} &= \begin{pmatrix} \frac{n_3 n_1^2 + n_2^2}{n_1^2 + n_2^2} & \frac{n_1 n_2 (n_3 - 1)}{n_1^2 + n_2^2} & n_1 \\ \frac{n_1 n_2 (n_3 - 1)}{n_1^2 + n_2^2} & \frac{n_1^2 + n_3 n_2^2}{n_1^2 + n_2^2} & n_2 \\ -n_1 & -n_2 & n_3 \end{pmatrix}.
\end{aligned}$$

$$S2F = \begin{pmatrix} \frac{n_3 n_1^2 + n_2^2}{n_1^2 + n_2^2} & \frac{n_1 n_2 (n_3 - 1)}{n_1^2 + n_2^2} & -n_1 \\ \frac{n_1 n_2 (n_3 - 1)}{n_1^2 + n_2^2} & \frac{n_1^2 + n_3 n_2^2}{n_1^2 + n_2^2} & -n_2 \\ n_1 & n_2 & n_3 \end{pmatrix}$$

$$\text{currentspeed} = S2F \text{velocity};$$

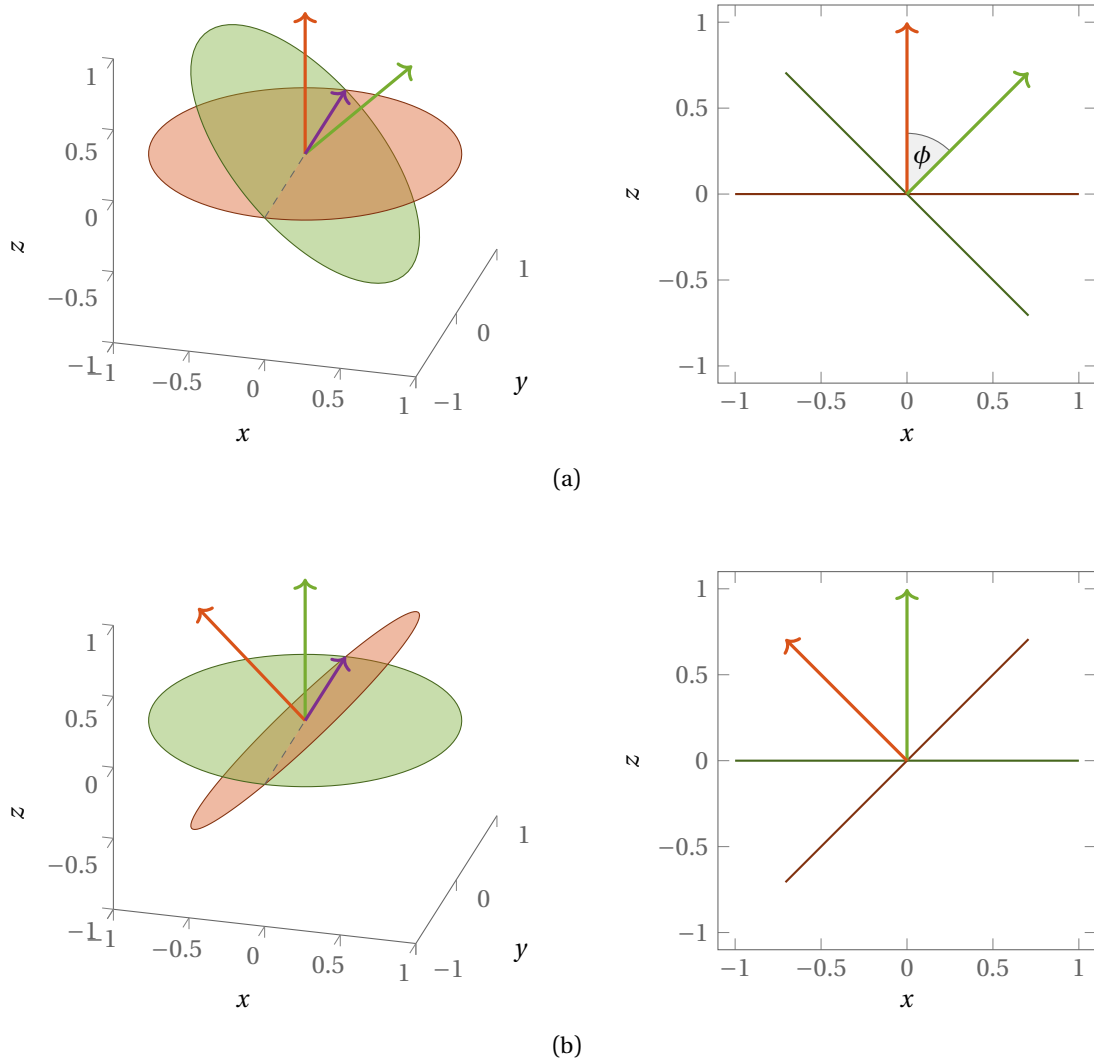


Figure 6: The unit vector $\hat{u}$ ($\rightarrow$) from equation (16). (a) Original setup: the key plane (red) and the surface (green). (b) Change of basis, with the surface normal $\hat{n}$ ($\rightarrow$) straight up.

2.4 CPM air control

Forwardkey aircontrol, forwardkey CGazHUD strafing, forwardkey side speed gain

Sidestrafe aircontrol applies regular acceleration with an additional change of direction. The new velocity is pulled towards the yaw angle.

TODO

2.5 Speed gains from air strafing

Acceleration from strafing can be obtained using equation (4). From below code, acceleration is applied per axis in direction of unit wishdir with magnitude of accelspeed, i.e. $\hat{s}_x a$ and $\hat{s}_y a$.

```
256 for (i=0 ; i<3 ; i++) {
257     pm->ps->velocity[i] += accelspeed*wishdir[i];
258 }
```

Listing 2: Code snippet from `code/game/bg_pmove.c`.

Thus, acceleration is dependent on angle δ .

W.l.o.g., assume current velocity $\underline{v}$ is solely on the x -axis. Then the resulting new velocity $\mathbf{r}$ becomes

$$\mathbf{r} = \begin{pmatrix} \underline{v} + \underline{a} \cos \delta \\ \underline{a} \sin \delta \\ 0 \end{pmatrix}.$$

Given $\delta \in [\delta_{\text{opt}}, 2\pi - \delta_{\text{opt}}]$, function of new velocity⁹ $r(\underline{v}; \delta)$ is given by

$$\begin{aligned} r(\underline{v}; \delta) &= \sqrt{(\underline{v} + \underline{a} \cos \delta)^2 + (\underline{a} \sin \delta)^2} \\ &= \sqrt{\underline{v}^2 + 2\underline{v}\underline{a} \cos \delta + \underline{a}^2} \end{aligned}$$

and likewise gain in speed $G(\underline{v}; \delta) = \sqrt{\underline{v}^2 + 2\underline{v}\underline{a} \cos \delta + \underline{a}^2} - \underline{v}$.

Now provided that the player is strafing at δ_{opt} ,

$$\begin{aligned} r(\underline{v}; \delta_{\text{opt}}) &= \sqrt{\underline{v}^2 + 2\underline{v}\underline{a} \cos \delta_{\text{opt}} + \underline{a}^2} \\ &= \sqrt{\underline{v}^2 + 2\underline{v}\underline{a} \frac{s - \underline{a}}{\underline{v}} + \underline{a}^2} \dots \frac{s - \underline{a}}{\underline{v}} \leq 1 \\ \therefore r(\underline{v}; \delta_{\text{opt}}) &= \sqrt{\underline{v}^2 + 2s\underline{a} - \underline{a}^2} \dots \underline{v} \geq s - \underline{a}. \end{aligned} \tag{17}$$

For normal strafing, $s = 320$, $A = 1$. These values are the same for CPM and VQ3. Using (17),

$$\begin{aligned} \underline{a} &= 320 \cdot 1 \cdot 0.008 = 2.56, & r_{\text{air-strafe}}(\underline{v}; \delta_{\text{opt}}) &= \sqrt{\underline{v}^2 + 2 \cdot 320 \cdot 2.56 - 2.56^2} \\ & & &= \sqrt{\underline{v}^2 + 1631.8464} \dots \underline{v} \geq 317.44. \end{aligned}$$

For CPM sidestrafing, $s = 30$, $A = 70$.

$$\begin{aligned} \underline{a} &= 30 \cdot 70 \cdot 0.008 = 16.8, & r_{\text{air-sidestrafe}}(\underline{v}; \delta_{\text{opt}}) &= \sqrt{\underline{v}^2 + 2 \cdot 30 \cdot 16.8 - 16.8^2} \\ & & &= \sqrt{\underline{v}^2 + 725.76} \dots \underline{v} \geq 13.2. \end{aligned}$$

This also presents the question of which method of strafing in CPM is superior. To answer this we can say that normal strafing for $\underline{v} < 317.44$ occurs at the optimal angle of $\delta_{\text{opt}} = 0$, and sidestrafing for $\underline{v} < 13.2$ occurs again at 0.

$$\begin{aligned} r_{\text{air-strafe}}(\underline{v}; \delta_{\text{opt}}) &= \begin{cases} \sqrt{\underline{v}^2 + 1631.8464}, & \underline{v} \geq 317.44 \\ \underline{v} + 2.56, & \underline{v} < 317.44 \end{cases} \\ r_{\text{air-sidestrafe}}(\underline{v}; \delta_{\text{opt}}) &= \begin{cases} \sqrt{\underline{v}^2 + 725.76}, & \underline{v} \geq 13.2 \\ \underline{v} + 16.8, & \underline{v} < 13.2 \end{cases} \end{aligned}$$

These functions intersect when $\underline{v} = 140.47$. Therefore, normal strafing is superior to sidestrafing except for when $\underline{v} < 140.47$ ups.

2.6 Speed gains from ground strafing

Deriving formulas for ground strafing works similarly to air strafing, however friction needs to be accounted for. As friction is applied before acceleration, $\underline{v}_f = (1 - \iota c T)\underline{v}$. Thus, assuming w.l.o.g. $\underline{v}$ is

⁹When strafing at angle δ to $\underline{v}$ for a single frame.

solely on the x -axis,

$$\mathbf{r} = \begin{pmatrix} (1 - \iota c T)\underline{v} + \underline{a} \cos \delta \\ \underline{a} \sin \delta \\ 0 \end{pmatrix}.$$

Then with $\delta \in [\delta_{\text{opt}}, 2\pi - \delta_{\text{opt}}]$,

$$r(\underline{v}; \delta) = \sqrt{((1 - \iota c T)\underline{v})^2 + 2(1 - \iota c T)\underline{v}\underline{a} \cos \delta + \underline{a}^2}$$

and likewise gain in speed $G(\underline{v}; \delta) = \sqrt{((1 - \iota c T)\underline{v})^2 + 2(1 - \iota c T)\underline{v}\underline{a} \cos \delta + \underline{a}^2} - \underline{v}$.

Now provided that the player is strafing at δ_{opt} ,

$$r(\underline{v}; \delta_{\text{opt}}) = \sqrt{((1 - \iota c T)\underline{v})^2 + 2s\underline{a} - \underline{a}^2} \dots \underline{v}_f \geq s - \underline{a}. \quad (18)$$

For CPM,

$$\underline{a} = 320 \cdot 15 \cdot 0.008 = 38.4, \quad r_{\text{CPM-ground-strafe}}(\underline{v}; \delta_{\text{opt}}) = \sqrt{((1 - \iota c T)\underline{v})^2 + 23101.44} \dots \underline{v}_f \geq 281.6.$$

For VQ3,

$$\underline{a} = 320 \cdot 10 \cdot 0.008 = 25.6, \quad r_{\text{VQ3-ground-strafe}}(\underline{v}; \delta_{\text{opt}}) = \sqrt{((1 - \iota c T)\underline{v})^2 + 15728.64} \dots \underline{v}_f \geq 294.4.$$

(18) can also be used to derive $\underline{v}_{\text{max}}$ from (11), done in Appendix C.6.1.

2.7 Swimming and flying

After friction is applied, the current velocity vector becomes

$$\mathbf{v}_f = \begin{pmatrix} v_{fx} \\ v_{fy} \\ v_{fz} \end{pmatrix} = \|\mathbf{v}_f\| \hat{\mathbf{v}} = (1 - cT)\mathbf{v},$$

with a magnitude

$$\|\mathbf{v}_f\| = \sqrt{\mathbf{v}_f^T \mathbf{v}_f} = \sqrt{v_{fx}^2 + v_{fy}^2 + v_{fz}^2} = v_f = (1 - cT)v,$$

with $c = 3$ the friction factor when using flight (`pm_flightfriction`).

$$\text{fm} = \|\text{fm}\| \hat{\text{fm}} = \text{fm} \begin{pmatrix} \cos \rho \cos \gamma \\ \cos \rho \sin \gamma \\ -\sin \rho \end{pmatrix},$$

$$\text{rm} = \|\text{rm}\| \hat{\text{rm}} = \text{rm} \begin{pmatrix} \sin \gamma \\ -\cos \gamma \\ 0 \end{pmatrix},$$

$$\text{um} = \|\text{um}\| \hat{\text{um}} = \text{um} \begin{pmatrix} \sin \rho \cos \gamma \\ \sin \rho \sin \gamma \\ \cos \rho \end{pmatrix},$$

$$\text{wishvel} = \text{fm} + \text{rm} + \text{um} \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} = \begin{pmatrix} \text{fm} \cos \rho \cos \gamma + \text{rm} \sin \gamma \\ \text{fm} \cos \rho \sin \gamma - \text{rm} \cos \gamma \\ -\text{fm} \sin \rho + \text{um} \end{pmatrix},$$

$$\text{fm} + \text{rm} + \text{um} = \begin{pmatrix} (\text{fm} \cos \rho + \text{um} \sin \rho) \cos \gamma + \text{rm} \sin \gamma \\ (\text{fm} \cos \rho + \text{um} \sin \rho) \sin \gamma - \text{rm} \cos \gamma \\ -\text{fm} \sin \rho + \text{um} \cos \rho \end{pmatrix},$$

$$\begin{aligned}\|\text{wishvel}\| &= \sqrt{(\text{fm} \cos \rho)^2 + \text{rm}^2 + (-\text{fm} \sin \rho + \text{um})^2} = \sqrt{\text{fm}^2 + \text{rm}^2 + \text{um}^2 - 2 \text{fm} \text{um} \sin \rho}, \\ \|\text{fm} + \text{rm} + \text{um}\| &= \sqrt{(\text{fm} \cos \rho + \text{um} \sin \rho)^2 + \text{rm}^2 + (-\text{fm} \sin \rho + \text{um} \cos \rho)^2} = \sqrt{\text{fm}^2 + \text{rm}^2 + \text{um}^2} = \|\text{cmd}\|,\end{aligned}$$

Using the law of cosines $\text{fum}^2 = \text{fm}^2 + \text{um}^2 - 2 \text{fm} \text{um} \cos\left(\frac{\pi}{2} - \rho\right)$ and

$$\|\text{wishvel}\| = \sqrt{\text{fum}^2 + \text{rm}^2},$$

$$\text{wishdir} = \hat{\mathbf{a}} = \frac{\text{wishvel}}{\|\text{wishvel}\|} = \frac{1}{\sqrt{\text{fm}^2 + \text{rm}^2 + \text{um}^2 - 2 \text{fm} \text{um} \sin \rho}} \begin{pmatrix} \text{fm} \cos \rho \cos \gamma + \text{rm} \sin \gamma \\ \text{fm} \cos \rho \sin \gamma - \text{rm} \cos \gamma \\ -\text{fm} \sin \rho + \text{um} \end{pmatrix},$$

$$\begin{aligned}\text{wishspeed} = s &= 320 \frac{\|\text{cmd}\|_\infty}{127} \frac{\|\text{wishvel}\|}{\|\text{cmd}\|} = 320 \frac{\|\text{cmd}\|_\infty}{127} \frac{\sqrt{\text{fm}^2 + \text{rm}^2 + \text{um}^2 - 2 \text{fm} \text{um} \sin \rho}}{\|\text{cmd}\|}, \\ \|\mathbf{a}\| = a &= sAT = 320AT \frac{\|\text{cmd}\|_\infty}{127} \frac{\|\text{wishvel}\|}{\|\text{cmd}\|},\end{aligned}$$

with $A = 8 (\text{pm_flyaccelerate})$.

However, in order to gain the full acceleration, a , we need the angle to be a bit larger, so we avoid the special case mentioned above. Therefore the optimal angle is the smallest angle at which we receive the full acceleration, a , which is given by

$$\begin{aligned}s(1 - AT) &= \mathbf{v}_f^T \hat{\mathbf{a}}, \\ 320 \frac{\|\text{cmd}\|_\infty}{127} \frac{\text{cmd}^2 - 2 \text{fm} \text{um} \sin \rho}{\text{cmd}} (1 - AT) &= v_{fx} (\text{fm} \cos \rho \cos \gamma + \text{rm} \sin \gamma) + v_{fz} (-\text{fm} \sin \rho + \text{um})\end{aligned}$$

$$A + B \sin \rho + C \sin \gamma + D \cos \rho \cos \gamma = 0, \quad (19)$$

$$\begin{aligned}S &= \frac{320 \|\text{cmd}\|_\infty (1 - AT)}{127}, \\ A &= S \text{cmd} - v_{fz} \text{um}, \\ B &= -\frac{2S}{\text{cmd}} \text{fm} \text{um} + v_{fz} \text{fm}, \\ C &= -v_{fx} \text{rm}, \\ D &= -v_{fx} \text{fm}, \\ \text{tmp} &= D^2 \cos^2(\gamma) - (A + C \sin(\gamma) - B)(A + C \sin(\gamma) + B); \\ \text{tmp}(\text{tmp} < 0) &= \text{nan}; \\ \text{tmp1} &= -2 \cdot \text{atan2}((B - \sqrt{\text{tmp}}), (A + C \sin(\gamma) - D \cos(\gamma))); \\ \text{tmp2} &= -2 \cdot \text{atan2}((B + \sqrt{\text{tmp}}), (A + C \sin(\gamma) - D \cos(\gamma))); \\ \text{tmp1}(\text{tmp1} > \pi) &= \text{tmp1}(\text{tmp1} > \pi) - 2\pi; \\ \text{tmp1}(\text{tmp1} < -\pi) &= \text{tmp1}(\text{tmp1} < -\pi) + 2\pi; \\ \text{tmp2}(\text{tmp2} > \pi) &= \text{tmp2}(\text{tmp2} > \pi) - 2\pi; \\ \text{tmp2}(\text{tmp2} < -\pi) &= \text{tmp2}(\text{tmp2} < -\pi) + 2\pi;\end{aligned}$$

$$\begin{aligned}\|\mathbf{v}_f + \|\mathbf{a}\| \hat{\mathbf{a}}\|^2 &= \|\mathbf{v}\|^2, \\ 2\mathbf{a}^T \mathbf{v}_f + \mathbf{a}^T \mathbf{a} &= \mathbf{v}^T \mathbf{v} - \mathbf{v}_f^T \mathbf{v}_f, \\ 2\|\mathbf{a}\| \mathbf{v}_f^T \hat{\mathbf{a}} + \|\mathbf{a}\|^2 &= \|\mathbf{v}\|^2 - \|\mathbf{v}_f\|^2, \\ 320AT \frac{\|\text{cmd}\|_\infty}{127} \frac{1}{\text{cmd}} 2(v_{fx} (\text{fm} \cos \rho \cos \gamma + \text{rm} \sin \gamma) + v_{fz} (-\text{fm} \sin \rho + \text{um})) &+ \\ 320^2 A^2 T^2 \frac{\|\text{cmd}\|_\infty^2}{127^2} \frac{\text{cmd}^2 - 2 \text{fm} \text{um} \sin \rho}{\text{cmd}^2} &= v_x^2 + v_z^2 - v_{fx}^2 - v_{fz}^2,\end{aligned}$$

$$S' = \frac{320AT\|\text{cmd}\|_\infty}{127} = \frac{320\|\text{cmd}\|_\infty}{127} - S,$$

$$S = \frac{320\|\text{cmd}\|_\infty}{127} - S',$$

$$A' = \frac{2S'}{\text{cmd}} \nu_{fz} \text{um} + S'^2 - \nu_x^2 - \nu_z^2 + \nu_{fx}^2 + \nu_{fz}^2 = A - S \text{cmd} + \left(\frac{2S'}{\text{cmd}} + 1 \right) \nu_{fz} \text{um} + S'^2 - \nu_x^2 - \nu_z^2 + \nu_{fx}^2 + \nu_{fz}^2,$$

$$B' = -\frac{2S'}{\text{cmd}} \nu_{fz} \text{fm} - \frac{2S'^2}{\text{cmd}^2} \text{fm um},$$

$$C' = \frac{2S'}{\text{cmd}} \nu_{fx} \text{rm} = -\frac{2S'}{\text{cmd}} C,$$

$$D' = \frac{2S'}{\text{cmd}} \nu_{fx} \text{fm} = -\frac{2S'}{\text{cmd}} D,$$

$$S' = \frac{320AT\|\text{cmd}\|_\infty}{127},$$

$$S = \frac{320\|\text{cmd}\|_\infty}{127} - S',$$

$$-\frac{2S'}{\text{cmd}}$$

$$A' = -\nu_{fz} \text{um} - \frac{\text{cmd}}{2S'} (S'^2 - \nu_x^2 - \nu_z^2 + \nu_{fx}^2 + \nu_{fz}^2),$$

$$B' = -\frac{S'}{\text{cmd}} \text{fm um} + \nu_{fz} \text{fm},$$

$$C' = -\nu_{fx} \text{rm},$$

$$D' = -\nu_{fx} \text{fm},$$

$$\frac{d}{d\rho} \|\mathbf{v}_f + \|\mathbf{a}\| \hat{\mathbf{a}}\|^2 = 0,$$

$$\frac{d}{d\rho} 2\mathbf{a}^T \mathbf{v}_f + \frac{d}{d\rho} \mathbf{a}^T \mathbf{a} = 0,$$

$$\frac{d}{d\rho} 2\|\mathbf{a}\| \mathbf{v}_f^T \hat{\mathbf{a}} + \frac{d}{d\rho} \|\mathbf{a}\|^2 = 0,$$

$$\frac{d}{d\rho} 320AT \frac{\|\text{cmd}\|_\infty}{127} \frac{1}{\text{cmd}} 2(\nu_{fx} (\text{fm} \cos \rho \cos \gamma + \text{rm} \sin \gamma) + \nu_{fz} (-\text{fm} \sin \rho + \text{um})) +$$

$$\frac{d}{d\rho} 320^2 A^2 T^2 \frac{\|\text{cmd}\|_\infty^2}{127^2} \frac{\text{cmd}^2 - 2 \text{fm um} \sin \rho}{\text{cmd}^2} = 0,$$

$$\frac{d}{d\rho} 320AT \frac{\|\text{cmd}\|_\infty}{127} \frac{1}{\text{cmd}} 2(\nu_{fx} \text{fm} \cos \rho \cos \gamma - \nu_{fz} \text{fm} \sin \rho) - \frac{d}{d\rho} 320^2 A^2 T^2 \frac{\|\text{cmd}\|_\infty^2}{127^2} \frac{2 \text{fm um}}{\text{cmd}^2} \sin \rho = 0,$$

$$-320AT \frac{\|\text{cmd}\|_\infty}{127} \frac{1}{\text{cmd}} 2(\nu_{fx} \text{fm} \sin \rho \cos \gamma + \nu_{fz} \text{fm} \cos \rho) - 320^2 A^2 T^2 \frac{\|\text{cmd}\|_\infty^2}{127^2} \frac{2 \text{fm um}}{\text{cmd}^2} \cos \rho = 0,$$

$$A \sin \rho \cos \gamma + B \cos \rho = 0,$$

$$S = \frac{320AT\|\text{cmd}\|_\infty}{127},$$

$$A = -\frac{2S}{\text{cmd}} \nu_{fx} \text{fm},$$

$$B = -\frac{2S}{\text{cmd}} \nu_{fz} \text{fm} - \frac{2S^2}{\text{cmd}^2} \text{fm um},$$

$$\begin{aligned}
\frac{d}{d\rho} \|\mathbf{v}_f + \|\mathbf{a}\| \hat{\mathbf{a}}\|^2 &= 0, \\
\frac{d}{d\rho} 2\mathbf{a}^T \mathbf{v}_f + \frac{d}{d\rho} \mathbf{a}^T \mathbf{a} &= 0, \\
\frac{d}{d\rho} 2\|\mathbf{a}\| \mathbf{v}_f^T \hat{\mathbf{a}} + \frac{d}{d\rho} \|\mathbf{a}\|^2 &= 0, \\
\frac{d}{d\gamma} 320AT \frac{\|\text{cmd}\|_\infty}{127} \frac{1}{\text{cmd}} 2(v_{fx}(\text{fm} \cos \rho \cos \gamma + \text{rm} \sin \gamma) + v_{fz}(-\text{fm} \sin \rho + \text{um})) + \\
\frac{d}{d\gamma} 320^2 A^2 T^2 \frac{\|\text{cmd}\|_\infty^2}{127^2} \frac{\text{cmd}^2 - 2 \text{fm} \text{um} \sin \rho}{\text{cmd}^2} &= 0, \\
\frac{d}{d\gamma} 320AT \frac{\|\text{cmd}\|_\infty}{127} \frac{1}{\text{cmd}} 2(v_{fx}(\text{fm} \cos \rho \cos \gamma + \text{rm} \sin \gamma)) &= 0, \\
-320AT \frac{\|\text{cmd}\|_\infty}{127} \frac{1}{\text{cmd}} 2(v_{fx}(\text{fm} \cos \rho \sin \gamma - \text{rm} \cos \gamma)) &= 0,
\end{aligned}$$

$$A \cos \rho \sin \gamma + B \cos \gamma = 0,$$

$$S = \frac{320AT \|\text{cmd}\|_\infty}{127},$$

$$A = -\frac{2S}{\text{cmd}} v_{fx} \text{fm},$$

$$B = \frac{2S}{\text{cmd}} v_{fx} \text{rm},$$

$$B \cos \rho - D \sin \rho \cos \gamma = 0,$$

$$C \cos \gamma - D \cos \rho \sin \gamma = 0,$$

Grey area:

$$\begin{aligned}
s &= \mathbf{v}_f^T \hat{\mathbf{a}}, \\
320 \frac{\|\text{cmd}\|_\infty}{127} \frac{\text{cmd}^2 - 2 \text{fm} \text{um} \sin \rho}{\text{cmd}} &= v_{fx}(\text{fm} \cos \rho \cos \gamma + \text{rm} \sin \gamma) + v_{fz}(-\text{fm} \sin \rho + \text{um})
\end{aligned}
\tag{19}$$

$$S = \frac{320 \|\text{cmd}\|_\infty}{127},$$

$$A = S \text{cmd} - v_{fz} \text{um},$$

$$B = -\frac{2S}{\text{cmd}} \text{fm} \text{um} + v_{fz} \text{fm},$$

$$C = -v_{fx} \text{rm},$$

$$D = -v_{fx} \text{fm},$$

PM_Accelerate (wishdir, wishspeed, pm_flyaccelerate);

$$\begin{aligned}
& \left\| \mathbf{v}_f + \left(s - \mathbf{v}_f^T \hat{\mathbf{a}} \right) \hat{\mathbf{a}} \right\|^2 = \|\mathbf{v}\|^2, \\
& \left(\mathbf{v}_f + \left(s - \mathbf{v}_f^T \hat{\mathbf{a}} \right) \hat{\mathbf{a}} \right)^T \left(\mathbf{v}_f + \left(s - \mathbf{v}_f^T \hat{\mathbf{a}} \right) \hat{\mathbf{a}} \right) = \mathbf{v}^T \mathbf{v}, \\
& 2 \left(s - \mathbf{v}_f^T \hat{\mathbf{a}} \right) \hat{\mathbf{a}}^T \mathbf{v}_f + \left(s - \mathbf{v}_f^T \hat{\mathbf{a}} \right)^2 \hat{\mathbf{a}}^T \hat{\mathbf{a}} = \mathbf{v}^T \mathbf{v} - \mathbf{v}_f^T \mathbf{v}_f, \\
& 2 \left(s - \mathbf{v}_f^T \hat{\mathbf{a}} \right) \mathbf{v}_f^T \hat{\mathbf{a}} + \left(s - \mathbf{v}_f^T \hat{\mathbf{a}} \right)^2 = \|\mathbf{v}\|^2 - \|\mathbf{v}_f\|^2, \\
& s^2 - \left(\mathbf{v}_f^T \hat{\mathbf{a}} \right)^2 = v^2 - v_f^2, \\
& 320^2 \frac{\|\text{cmd}\|_\infty^2}{127^2} \frac{(\text{cmd}^2 - 2 \text{ fm um sin } \rho)^2}{\text{cmd}^2} - \\
& \left(v_{fx} (\text{fm cos } \rho \cos \gamma + \text{rm sin } \gamma) + v_{fz} (-\text{fm sin } \rho + \text{um}) \right)^2 = \left(v_x^2 + v_z^2 - v_{fx}^2 - v_{fz}^2 \right) (\text{cmd}^2 - 2 \text{ fm um sin } \rho),
\end{aligned}$$

which is too complex to calculate efficiently.

3 SnapHUD

3.1 Basics

In order to save resources, the game engine rounds each component of some vectors each frame, as shown by the below code. Albeit a simple change, this largely changes the way the game is played.

```
151 void Sys_SnapVector( float *v ) { // bk001213 - see win32/win_shared.c
152     // bk001213 - old linux
153     v[0] = rint(v[0]);
154     v[1] = rint(v[1]);
155     v[2] = rint(v[2]);
156 }
```

Listing 3: Code snippet from `code/unix/unix_shared.c`.

Since player velocity is one of the vectors that is rounded, we can interpret this as rounding the speed change the player receives each frame, forming

$$\lfloor \mathbf{a} \rfloor = \begin{pmatrix} \lfloor a_x \rfloor \\ \lfloor a_y \rfloor \\ \lfloor a_z \rfloor \end{pmatrix}, \quad \lfloor \mathbf{r} \rfloor = \begin{pmatrix} \lfloor r_x \rfloor \\ \lfloor r_y \rfloor \\ \lfloor r_z \rfloor \end{pmatrix}.$$

Hence, there are specific constant view yaw angle ranges called “snap zones” which provide the same acceleration regardless of where the player is looking in the zone. Some of these zones are rounded to give a value $\|\lfloor \underline{\mathbf{a}} \rfloor\| > sAT$, and similarly some are rounded such that $\|\lfloor \underline{\mathbf{a}} \rfloor\| < sAT$. Note that the player can only accelerate if $\delta \in [\delta_{\min}, 2\pi - \delta_{\min}]$.

Additionally, the closer $\delta_{\min}$ is to the border of a snap zone, there will be a greater increase in speed, as the acceleration is pointed more in the direction of $\underline{\mathbf{v}}$ meaning more of the acceleration goes into increasing the speed rather than changing the direction. This also means that as $\delta_{\min}$ gets closer to the border of a snap zone, $\delta_{\min}$ will change faster due to speed increasing more rapidly. Players often attempt to optimize (by decreasing) the distance between $\delta_{\min}$ and the border of the snap zone in order to maximize acceleration through techniques referred to as “snap manipulation” or “snap zone manipulation.”

Furthermore, due to velocity snapping rotating the acceleration vector, given a sufficiently high velocity it is possible to lose speed while strafing at δ_{opt} , which is discussed in Section 3.4.

We can form an illustration of the snap zones in any conditions given that we know the acceleration the player is receiving in that frame. In normal conditions while air strafing at 125 fps with no moveup and $\delta \in [\delta_{\text{opt}}, 2\pi - \delta_{\text{opt}}]$, $\underline{\mathbf{a}} = 2.56$. The effect of the rounding can be visualized with a circle with radius 2.56.

TODO

3.2 Frame rate impact

TODO

3.3 Proper maximum ground speed

The addition of velocity snapping also affects the maximum ground speed previously calculated to be $\underline{v}_{\max}$. There are technically two solutions to the proper maximum ground speed for CPM and VQ3: allowing the player to turn any direction (clockwise and anticlockwise), or restricting the player’s rotation to one direction as is done in a circle jump.

TODO: method

Results

Restricted	CPM results	Unrestricted
$\underline{v}_{\max_proper} = \left\ \begin{pmatrix} TODO \\ TODO \\ 0 \end{pmatrix} \right\ = 498.787 \text{ ups}$		$\underline{v}_{\max_proper} = \left\ \begin{pmatrix} TODO \\ TODO \\ 0 \end{pmatrix} \right\ = 500.3 \text{ ups}$
	VQ3 results	
$\underline{v}_{\max_proper} = \left\ \begin{pmatrix} TODO \\ TODO \\ 0 \end{pmatrix} \right\ = TODO$		$\underline{v}_{\max_proper} = \left\ \begin{pmatrix} 403 \\ 91 \\ 0 \end{pmatrix} \right\ = 413.146 \text{ ups}$

3.4 Speed loss while strafing optimally

As mentioned before, given a high enough velocity the player can lose speed while strafing with $\delta \in [\delta_{\text{opt}}, 2\pi - \delta_{\text{opt}}]$. CPM players often attempt to counteract this through snap zone manipulation, by W-turning through such areas of speed loss.

While air strafing, the lowest velocity that such speed loss can occur can be calculated. First it would be useful to define *symmetrical snap zones* as snap zones where the respective $\lfloor \underline{a} \rfloor$ vector lies in the middle of the snap zone borders. It does not matter which direction these zones are approached from. Similarly, *asymmetrical snap zones* are snap zones where this is not the case. There is 1 symmetrical snap zone when $\underline{a} < 0.5$ and 8 when $\underline{a} \geq 0.5$, logically being the snap zones along the axes and diagonals. The first asymmetrical snap zones appear after $\underline{a} \geq 1.5$. A loss in speed from strafing can first occur in a zone with the largest difference (in terms of angle) between a border of such snap zone and the respective $\lfloor \underline{a} \rfloor$ for that zone.

In normal conditions of 125 fps while air strafing with no upmove, this difference Δ is equal to

$$\text{atan}\left(\frac{1}{2}\right) - \text{atan}\left(\frac{\sqrt{\underline{a}^2 - 2.5^2}}{2.5}\right) = 0.246716 \text{ rad},$$

and occurs when entering the big uncoloured snap zone from the small zone,¹⁰ and hence $\|\lfloor \underline{a} \rfloor\| = \sqrt{1^2 + 2^2} = \sqrt{5}$. Since velocity snapping changes the received acceleration and a proper $\delta_{\max}$ value depends on the acceleration after velocity snapping, we must first define $\delta_{\max_proper}$ as

$$\begin{aligned} \delta_{\max_proper} &= \arccos\left(\frac{\underline{v}^2 - \underline{v}_f^2 - \|\lfloor \underline{a} \rfloor\|^2}{2\|\lfloor \underline{a} \rfloor\|\underline{v}_f}\right) \\ &= \arccos\left(-\frac{\|\lfloor \underline{a} \rfloor\|}{2\underline{v}}\right) \dots \text{ while in air.} \end{aligned} \tag{20}$$

Note that definitions of all other proper δ variables are situation-based so a formula similar to this cannot realistically be derived. Assuming the player is strafing at δ_{opt} , velocities at which speed can be lost by strafing can hence be calculated by solving

$$\delta_{\text{opt}} + \Delta > \delta_{\max_proper}$$

$$\arccos\left(\frac{s - \underline{a}}{\underline{v}}\right) + \Delta > \arccos\left(-\frac{\|\lfloor \underline{a} \rfloor\|}{2\underline{v}}\right),$$

giving $\underline{v} > 1304.247$ ups, which first occurs with

$$\underline{v} = \begin{pmatrix} 583 \\ -1169 \\ v_z \end{pmatrix}.$$

It should be noted this solution is frame rate dependent, and that the solution can be reflected about the x and y axes and diagonals to produce 7 other equivalent solutions.

¹⁰TODO: explain each of the 24 snap zones

4 Miscellaneous

Features/Phenomenon which didn't fit into any other category.

4.1 Influence of movement keys

$$\text{cmd} = \begin{pmatrix} \text{fm} \\ \text{rm} \\ \text{um} \end{pmatrix}, \quad \underline{\text{cmd}} = \begin{pmatrix} \text{fm} \\ \text{rm} \\ 0 \end{pmatrix},$$

The direction of acceleration $\underline{\hat{a}}$ (3) can be chosen by using your mouse and the movement keys. Relative to $v = \text{atan}\left(\frac{v_y}{v_x}\right)$

$$\underline{\hat{v}} = \frac{1}{v} \begin{pmatrix} v_x \\ v_y \\ 0 \end{pmatrix}$$

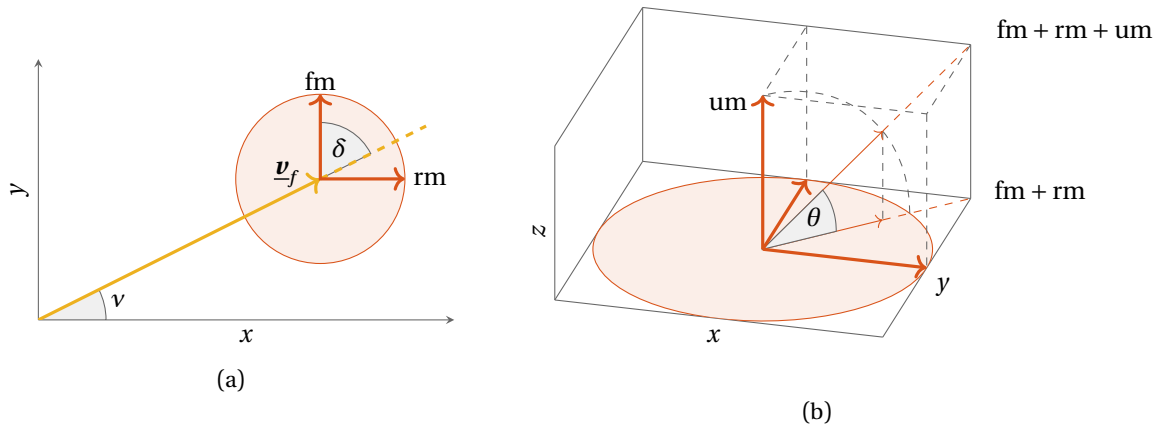


Figure 7: The current velocity $\underline{v}_f$ (→), the acceleration $\underline{a}$ (→) and the resulting new velocity $\underline{r}$ (→). (a) is the top view of (b). key plane (□). $\theta = \text{atan}\sqrt{2}$

where $\gamma = \delta + v$ represents the yaw angle.

Appendix B.1

The current velocity vector is defined as

$$\begin{aligned} \hat{\text{fm}} &= \begin{pmatrix} \cos \rho \cos \gamma \\ \cos \rho \sin \gamma \\ -\sin \rho \end{pmatrix}, \\ \hat{\text{rm}} &= \begin{pmatrix} -\sin \sigma \sin \rho \cos \gamma + \cos \sigma \sin \gamma \\ -\sin \sigma \sin \rho \sin \gamma - \cos \sigma \cos \gamma \\ -\sin \sigma \cos \rho \end{pmatrix}, \\ \hat{\text{um}} &= \begin{pmatrix} \cos \sigma \sin \rho \cos \gamma + \sin \sigma \sin \gamma \\ \cos \sigma \sin \rho \sin \gamma - \sin \sigma \cos \gamma \\ \cos \sigma \cos \rho \end{pmatrix}, \end{aligned}$$

```
621 // project moves down to flat plane
622 pml.forward[2] = 0;
623 pml.right[2] = 0;
624 VectorNormalize (pml.forward);
625 VectorNormalize (pml.right);
```

Listing 4: Code snippet from `code/game/bg_pmove.c`.

$$\underline{\hat{\mathbf{f}}\mathbf{m}} = \begin{pmatrix} \cos \rho \cos \gamma \\ \cos \rho \sin \gamma \\ 0 \end{pmatrix} \frac{1}{\sqrt{\cos^2 \rho}} = \begin{pmatrix} \cos \gamma \\ \sin \gamma \\ 0 \end{pmatrix},$$

$$\underline{\hat{\mathbf{r}}\mathbf{m}} = \begin{pmatrix} -\sin \sigma \sin \rho \cos \gamma + \cos \sigma \sin \gamma \\ -\sin \sigma \sin \rho \sin \gamma - \cos \sigma \cos \gamma \\ 0 \end{pmatrix} \frac{1}{\sqrt{\sin^2 \sigma \sin^2 \rho + \cos^2 \sigma}},$$

$$\begin{aligned} \underline{\hat{\mathbf{f}}\mathbf{m}}^T \underline{\hat{\mathbf{r}}\mathbf{m}} &= \frac{-\sin \sigma \sin \rho \cos^2 \gamma + \cos \sigma \sin \gamma \cos \gamma - \sin \sigma \sin \rho \sin^2 \gamma - \cos \sigma \sin \gamma \cos \gamma}{\sqrt{\sin^2 \sigma \sin^2 \rho + \cos^2 \sigma}} \\ &= \frac{-\sin \sigma \sin \rho}{\sqrt{\sin^2 \sigma \sin^2 \rho + \cos^2 \sigma}} \end{aligned}$$

$$\underline{\mathbf{f}}\mathbf{m} + \underline{\mathbf{r}}\mathbf{m} = \text{wishvel} = \mathbf{f}\mathbf{m} \begin{pmatrix} \cos \gamma \\ \sin \gamma \\ 0 \end{pmatrix} + \mathbf{r}\mathbf{m} \begin{pmatrix} -\sin \sigma \sin \rho \cos \gamma + \cos \sigma \sin \gamma \\ -\sin \sigma \sin \rho \sin \gamma - \cos \sigma \cos \gamma \\ 0 \end{pmatrix} \frac{1}{\sqrt{\sin^2 \sigma \sin^2 \rho + \cos^2 \sigma}},$$

$$\begin{aligned} \|\underline{\mathbf{f}}\mathbf{m} + \underline{\mathbf{r}}\mathbf{m}\|^2 &= \|\text{wishvel}\|^2 = \mathbf{f}\mathbf{m}^2 \cos^2 \gamma + \mathbf{r}\mathbf{m}^2 \frac{\sin^2 \sigma \sin^2 \rho \cos^2 \gamma + \cos^2 \sigma \sin^2 \gamma - \sin \sigma \cos \sigma \sin \rho \sin \gamma \cos \gamma}{\sin^2 \sigma \sin^2 \rho + \cos^2 \sigma} + \\ &\quad \mathbf{f}\mathbf{m}^2 \sin^2 \gamma + \mathbf{r}\mathbf{m}^2 \frac{\sin^2 \sigma \sin^2 \rho \sin^2 \gamma + \cos^2 \sigma \cos^2 \gamma + \sin \sigma \cos \sigma \sin \rho \sin \gamma \cos \gamma}{\sin^2 \sigma \sin^2 \rho + \cos^2 \sigma} + \\ &\quad 2 \mathbf{f}\mathbf{m} \mathbf{r}\mathbf{m} \frac{-\sin \sigma \sin \rho \cos^2 \gamma + \cos \sigma \sin \gamma \cos \gamma}{\sqrt{\sin^2 \sigma \sin^2 \rho + \cos^2 \sigma}} + \\ &\quad 2 \mathbf{f}\mathbf{m} \mathbf{r}\mathbf{m} \frac{-\sin \sigma \sin \rho \sin^2 \gamma - \cos \sigma \sin \gamma \cos \gamma}{\sqrt{\sin^2 \sigma \sin^2 \rho + \cos^2 \sigma}}, \\ &= \mathbf{f}\mathbf{m}^2 + \mathbf{r}\mathbf{m}^2 + 2 \mathbf{f}\mathbf{m} \mathbf{r}\mathbf{m} \frac{-\sin \sigma \sin \rho}{\sqrt{\sin^2 \sigma \sin^2 \rho + \cos^2 \sigma}}, \end{aligned}$$

Derivative

$$\frac{d}{d\rho} \|\underline{\mathbf{f}}\mathbf{m} + \underline{\mathbf{r}}\mathbf{m}\|^2 = \frac{d}{d\rho} \|\text{wishvel}\|^2 = 2 \mathbf{f}\mathbf{m} \mathbf{r}\mathbf{m} \frac{-\sin \sigma \cos^2 \sigma \cos \rho}{(\sin^2 \sigma \sin^2 \rho + \cos^2 \sigma)^{3/2}},$$

This is 0 when

$$\begin{cases} \rho = \frac{\pi}{2} + 2k\pi, & k \in \mathbb{N}, \\ \rho = -\frac{\pi}{2} + 2k\pi, & k \in \mathbb{N}. \end{cases}$$

With optimal ρ filled in, we get

$$\begin{aligned} \underline{\hat{\mathbf{f}}\mathbf{m}}^T \underline{\hat{\mathbf{r}}\mathbf{m}} &= -\sin \sigma, \\ &= \sin \sigma, \end{aligned}$$

$$\begin{aligned} \|\underline{\mathbf{f}}\mathbf{m} + \underline{\mathbf{r}}\mathbf{m}\|^2 &= \|\text{wishvel}\|^2 = \mathbf{f}\mathbf{m}^2 + \mathbf{r}\mathbf{m}^2 - 2 \mathbf{f}\mathbf{m} \mathbf{r}\mathbf{m} \sin \sigma, \\ &= \mathbf{f}\mathbf{m}^2 + \mathbf{r}\mathbf{m}^2 + 2 \mathbf{f}\mathbf{m} \mathbf{r}\mathbf{m} \sin \sigma, \end{aligned}$$

Appendix B.3

$$\text{scale} = \frac{\text{speed} \|\text{cmd}\|_\infty}{127 \|\text{cmd}\|},$$

$$\text{wishspeed} = \text{scale} \|\text{wishvel}\| = \frac{\text{speed} \|\text{cmd}\|_{\infty}}{127 \|\text{cmd}\|} \|\text{wishvel}\|,$$

$$\|\underline{a}\| = \underline{a} = A T \text{wishspeed} = A T \frac{\text{speed} \|\text{cmd}\|_{\infty}}{127 \|\text{cmd}\|} \|\text{wishvel}\|,$$

when $\text{fm} = 127$, $\text{rm} = 0$ and $\text{um} = 0$, then $s = 320$ like in the previous section.

Air:

$$\begin{aligned} \|\underline{P_a} \underline{v_f}\| + \|\underline{a}\| &= s \|\underline{a}\|, \\ \underline{v_f}^T \text{wishdir} + \text{accel} T \text{wishspeed} &= \text{wishspeed} \|\text{wishdir}\|, \\ \|\underline{v_f}\| \cos \delta_{\text{opt}} + \text{accel} T \text{wishspeed} &= \text{wishspeed}, \\ \underline{v_f} \cos \delta_{\text{opt}} &= \text{wishspeed} - \text{accel} T \text{wishspeed}, \\ \delta_{\text{opt}} &= \arccos \left(\frac{\text{wishspeed} (1 - \text{accel} T)}{\underline{v_f}} \right). \end{aligned} \quad (21)$$

PM_Accelerate (wishdir, wishspeed, pm_airaccelerate);

The strafe keys in CPM do not ever affect upmove. Not true

4.2 Upmove effect on acceleration

```

615 cmd = pm->cmd;
616 scale = PM_CmdScale( &cmd );
617
618 // set the movementDir so clients can rotate the legs for strafing
619 PM_SetMovementDir();
620
621 // project moves down to flat plane
622 pml.forward[2] = 0;
623 pml.right[2] = 0;
624 VectorNormalize (pml.forward);
625 VectorNormalize (pml.right);
626
627 for ( i = 0 ; i < 2 ; i++ ) {
628     wishvel[i] = pml.forward[i]*fmove + pml.right[i]*smove;
629 }
630 wishvel[2] = 0;
631
632 VectorCopy (wishvel, wishdir);
633 wishspeed = VectorNormalize(wishdir);
634 wishspeed *= scale;
```

Listing 5: Code snippet from `code/game/bg_pmove.c`.

From the above code listing, the current velocity is projected down to the xy -plane, is normalized, and then a scale factor is applied forming wishspeed (s). This scale factor is determined by the below code.

```

306 total = sqrt( cmd->forwardmove * cmd->forwardmove
307             + cmd->rightmove * cmd->rightmove + cmd->upmove * cmd->upmove );
308 scale = (float)pm->ps->speed * max / ( 127.0 * total );
```

Listing 6: Code snippet from `code/game/bg_pmove.c`.

Hence upmove increases the denominator of the scale factor, decreasing wishspeed.

If the player is holding 3 movement keys, (fm , rm and um), this transformation scales it by a factor of $\frac{\sqrt{2}}{\sqrt{3}}$.

If the player is holding 2 movement keys, (fm and um or rm and um), the scale factor is $\frac{\sqrt{2}}{2}$.

Hence, the snap zones will change along with all CGazHUD δ angles, which explains the altered acceleration while holding moveup or movedown. Also, holding 2 movement keys reduces wishspeed moreso

than holding 3 movement keys, decreasing acceleration more¹¹.

4.3 Frame rate dependency

We can define *frame rate dependency* as attributes of the game engine that act differently depending on frame rate, and say that an attribute of the game engine has *frame independency* if the time between frames does not need to be equidistant for the physics to act the same.

Given that acceleration is impacted directly by frame rate (by $\underline{a} = sAT$), frame rate has a large impact on speed gains from strafing.

TODO: frame rate effect on strafing

Additionally, T is floored to the nearest millisecond (0.001), giving us proper- T T_p , and frames are delayed until $1000T_p$ milliseconds pass. For example, with frame rate 140 fps, T would be floored from $1/140 = 0.007143$ to 0.007. Note: minMsec is an int.

```
2686 if ( !com_dedicated->integer && com_maxfps->integer > 0 && !com_timedemo->integer ) {
2687     minMsec = 1000 / com_maxfps->integer;
2688 } else {
2689     minMsec = 1;
2690 }
```

Listing 7: Code snippet from `code/qcommon/common.c`.

```
2599 if ( msec < 1 && com_timescale->value ) {
2600     msec = 1;
2601 }
2602
2603 if ( com_dedicated->integer ) {
2604     // dedicated servers don't want to clamp for a much longer
2605     // period, because it would mess up all the client's views
2606     // of time.
2607     if ( msec > 500 ) {
2608         Com_Printf( "Hitch warning: %i msec frame time\n", msec );
2609     }
2610     clampTime = 5000;
2611 } else
2612 if ( !com_sv_running->integer ) {
2613     // clients of remote servers do not want to clamp time, because
2614     // it would skew their view of the server's time temporarily
2615     clampTime = 5000;
2616 } else {
2617     // for local single player gaming
2618     // we may want to clamp the time to prevent players from
2619     // flying off edges when something hitches.
2620     clampTime = 200;
2621 }
2622
2623 if ( msec > clampTime ) {
2624     msec = clampTime;
2625 }
```

Listing 8: Code snippet from `code/qcommon/common.c`.

Hence,

$$T_p = \min \left(5, \max \left(0.001, \frac{\lfloor 1000T \rfloor}{1000} \right) \right), \quad \text{when running a dedicated server or client of a remote server}$$

$$T_p = \min \left(0.2, \max \left(0.001, \frac{\lfloor 1000T \rfloor}{1000} \right) \right), \quad \text{otherwise.}$$

¹¹Thus, although insignificant, full-beat VQ3 strafing is superior to half-beat and moreso to invert.

Also, frame rate impacts vertical (z -axis) velocity when in air, shown by

$$\begin{aligned} r_z &= v_z - g T_p & \dots \quad g &= 800, \\ \lfloor r_z \rfloor &= v_z - \lfloor g T_p \rfloor & \dots \quad v_z &= \lfloor v_z \rfloor \text{ when in air} \end{aligned}$$

Hence, with $T = 1/125 = T_p$,

$$\begin{aligned} r_z &= v_z - 6.4, \\ \lfloor r_z \rfloor &= v_z - 6, \end{aligned}$$

and thus over one second v_z is reduced by 750 ups instead of 800 ups, allowing the player to jump higher and hence further. As a general rule, if effective-gravity $\frac{\lfloor g T_p \rfloor}{T_p} < 800$ then frame rates given by $\frac{1}{T}$ allow the player to jump higher than intended. This is expanded upon in Section 4.4 (TODO).

Another important point to note is that accelerations on one axis a_n can be rounded to 0 if $a_n \leq 0.5$. This ironically places limits on the functionality of the movement physics at high frame rates, as accelerations will be more frequently nulled.

Frame rate dependency with respect to overbounces is covered in Section 4.7 (TODO).

4.4 Approach to z -axis translations

The game engine applies vertical velocity in a slightly unintuitive way in order to make all z -axis position translations frame independent. In regards to the z -axis, new velocity is calculated using acceleration, velocity is to player position, and then velocity is snapped. However the key here is *how* velocity is applied to the player position, demonstrated in (23).

Defining $v_{z,k}$ as the sequence of z -axis velocities after snapping after k frames
and $x_{z,k}$ as the sequence of the player's z -axis position after k frames,

$$v_{z,k+1} = v_{z,k} - \lfloor g T_p \rfloor \tag{22}$$

$$x_{z,k+1} = x_{z,k} + \left(v_{z,k} - \frac{g T_p}{2} \right) T_p \tag{23}$$

$$\begin{aligned} v_{z,1} &= v_{z,0} - \lfloor g T_p \rfloor \\ v_{z,2} &= v_{z,1} - \lfloor g T_p \rfloor \\ &= v_{z,0} - 2 \lfloor g T_p \rfloor \\ \therefore v_{z,k} &= v_{z,0} - k \lfloor g T_p \rfloor. \end{aligned} \tag{24}$$

Same approach is used to derive $x_{z,k}$, done in Appendix C.7, yielding

$$\begin{aligned} x_{z,k} &= x_{z,0} + k \left(v_{z,0} + \frac{\lfloor g T_p \rfloor - g T_p}{2} \right) T_p - \frac{k^2}{2} \lfloor g T_p \rfloor T_p \\ &= x_{z,0} + k T_p \left(v_{z,0} + \frac{(1-k) \lfloor g T_p \rfloor - g T_p}{2} \right). \end{aligned} \tag{25}$$

To solve for the frame n_{peak} that you reach the peak, differentiate $x_{z,k}$.

$$\begin{aligned}
x'_{z,k} &= \left(v_{z,0} + \frac{\lfloor g T_p \rfloor - g T_p}{2} \right) T_p - k \lfloor g T_p \rfloor T_p \\
0 &= \left(v_{z,0} + \frac{\lfloor g T_p \rfloor - g T_p}{2} \right) T_p - k \lfloor g T_p \rfloor T_p \\
\Rightarrow k &= \frac{v_{z,0} - \frac{g T_p}{2} + \frac{\lfloor g T_p \rfloor}{2}}{\lfloor g T_p \rfloor} \\
&= \frac{v_{z,0} - \frac{g T_p}{2}}{\lfloor g T_p \rfloor} + \frac{1}{2} \\
\Rightarrow n_{peak} &= \left\lceil \frac{v_{z,0} - \frac{g T_p}{2}}{\lfloor g T_p \rfloor} + \frac{1}{2} \right\rceil \\
\therefore n_{peak} &= \left\lceil \frac{v_{z,0} - \frac{g T_p}{2}}{\lfloor g T_p \rfloor} \right\rceil
\end{aligned} \tag{26}$$

With 125 fps and jump velocity $v_{z,0} = 270$ ups, the peak is reached after $n_{peak} = 45$ frames.

The maximum height the player reaches in a jump is given by substituting n_{peak} into $x_{z,k}$, with $x_{z,0} = 0$. When step-up is introduced in Section 4.5 this formula changes.

$$\left\lceil \frac{v_{z,0} - \frac{g T_p}{2}}{\lfloor g T_p \rfloor} \right\rceil \left(v_{z,0} + \frac{\lfloor g T_p \rfloor - g T_p}{2} \right) T_p - \frac{1}{2} \left\lceil \frac{v_{z,0} - \frac{g T_p}{2}}{\lfloor g T_p \rfloor} \right\rceil^2 \lfloor g T_p \rfloor T_p \tag{27}$$

This implementation of z-axis translations is frame independent, as shown below.

$$v_{z,k} = v_{z,0} - k \lfloor g T_p \rfloor \dots \tag{24}$$

Using trapezoidal rule,

$$\begin{aligned}
&\sum_{i=0}^{k-1} \left(\left(v_{z,0} - i \lfloor g T_p \rfloor \right) + \left(v_{z,0} - (i+1) \lfloor g T_p \rfloor \right) \right) \frac{T_p}{2} \\
&\sum_{i=0}^{k-1} \left(v_{z,0} - \left(i + \frac{1}{2} \right) \lfloor g T_p \rfloor \right) T_p \\
\Rightarrow x_{z,k} &= x_{z,0} + k v_{z,0} T_p - \frac{k^2}{2} \lfloor g T_p \rfloor T_p,
\end{aligned}$$

but $x_{z,k}$ is

$$x_{z,0} + k \left(v_{z,0} + \frac{\lfloor g T_p \rfloor - g T_p}{2} \right) T_p - \frac{k^2}{2} \lfloor g T_p \rfloor T_p \dots \tag{25}.$$

Using inverse trapezoidal rule,

$$\begin{aligned}
&\sum_{i=0}^{k-1} \left(\left(v_{z,0} + \frac{\lfloor g T_p \rfloor - g T_p}{2} \right) - \left(i + \frac{1}{2} \right) \lfloor g T_p \rfloor \right) T_p \\
&\sum_{i=0}^{k-1} \left(\left(\left(v_{z,0} + \frac{\lfloor g T_p \rfloor - g T_p}{2} \right) - i \lfloor g T_p \rfloor \right) + \left(\left(v_{z,0} + \frac{\lfloor g T_p \rfloor - g T_p}{2} \right) - (i+1) \lfloor g T_p \rfloor \right) \right) \frac{T_p}{2} \\
\Rightarrow v_{z,k} &= \left(v_{z,0} + \frac{\lfloor g T_p \rfloor - g T_p}{2} \right) - k \lfloor g T_p \rfloor
\end{aligned} \tag{28}$$

Thus, z -axis position is updated using this “corrected” velocity, meaning z -axis translations are frame independent, since a fractional k would still result in the correct change in position. Hence $k \in \mathbb{R}^+$.

As x_z is updated using “corrected” velocity (28) rather than (24), the difference in applied velocity is

$$\begin{aligned} (28) - (24) &= v_{z,0} + \frac{\lfloor g T_p \rfloor - g T_p}{2} - k \lfloor g T_p \rfloor - v_{z,0} + k \lfloor g T_p \rfloor \\ &= \frac{\lfloor g T_p \rfloor - g T_p}{2}. \end{aligned}$$

This implementation results in a height “error” h , given by

$$h = n_{gravity} T_p \frac{\lfloor g T_p \rfloor - g T_p}{2} \dots \text{ where } n_{gravity} \in \mathbb{R}^+ \text{ is the number of frames that gravity was applied.} \quad (29)$$

For example, a jump at 125 fps lasts $n_{gravity} = 90$ frames, so the final x_z of the player is $h = -0.144$ units relative to the starting x_z ¹².

As a side note, this implementation of z -axis translations is also frame rate independent $\iff$ rounding is ignored, since

- Vertical acceleration is linear (constant) and thus
- Ignoring rounding,

$$\begin{aligned} x_{z,k} &= x_{z,0} + k \left(v_{z,0} + \frac{g T_p - g T_p}{2} \right) T_p - \frac{k^2}{2} g T_p T_p \\ &= x_{z,0} + k v_{z,0} T_p - \frac{k^2 g T_p^2}{2} \\ &= x_{z,0} + (k T_p) v_{z,0} - \frac{(k T_p)^2 g}{2}, \end{aligned}$$

which is the same as the physical model equation. Without ignoring rounding, the implementation is frame rate dependent, hence the differing jump heights by changing frame rate.

It can be shown that $T_p = 0.003$ results in the largest jump height, with effective-gravity of $666.\bar{6}$.
TODO

4.5 Step-up

TODO: explanation of step-up, conditions, special cases (if any)

The maximum height the player can step-up is 18 units. Hence, to calculate the maximum height the player can jump including step-up, add 18 units to (27) giving

$$\left\lceil \frac{v_{z,0} - \frac{g T_p}{2}}{\lfloor g T_p \rfloor} \right\rceil \left(v_{z,0} + \frac{\lfloor g T_p \rfloor - g T_p}{2} \right) T_p - \frac{1}{2} \left\lceil \frac{v_{z,0} - \frac{g T_p}{2}}{\lfloor g T_p \rfloor} \right\rceil^2 \lfloor g T_p \rfloor T_p + 18, \quad (30)$$

which is 66.528 units at 125 fps.

4.6 Skimming

TODO

¹²Since $h < 0$ the final position is below the starting position, assuming the player didn't clip into the ground plane.

4.7 Overbounces

TODO

4.8 Wallbug

Wallbugs occur due to z -axis velocity being zeroed when the game thinks the player entity is inside a solid when they actually aren't. Zeroing the z -axis velocity causes the player to stop falling, but still be able to accelerate by strafing along the xy -plane.

```
99 if (trace.allsolid) {  
100     // entity is completely trapped in another solid  
101     pm->ps->velocity[2] = 0;           // don't build up falling damage, but allow sideways acceleration  
102     return qtrue;  
103 }
```

Listing 9: Code snippet from `code/game/bg_slidemove.c`.

TODO: escape conditions

5 Weapons

TODO

6 RR ranking system

[ite21]

A good map score is created by having many strong competitors, small (your time)/(fastest time by someone else), and not placing under people, especially under weak opponents.

Your overall score is the sum of your 10 best individual map scores, plus 1/2 of the next 10 scores, plus 1/4 of the next 10 scores, and so on. You should focus on your top 50 maps, which add to 97% of your overall score.

A bad time will never decrease your score; it just won't affect it.

$$k \sim 1 \quad (31)$$

$$b \sim 5 \quad (32)$$

$$c \sim 0.1 \quad (33)$$

$$d \sim 1.1 \quad (34)$$

$$g \sim 0.9 \quad (35)$$

$$m \sim 10 \quad (36)$$

$$I_y = \text{player } y\text{'s overall score} = \frac{\sqrt[m]{2} - 1}{\sqrt[m]{2}} \sum_{x=0}^m 2^{-x/m} M_x \quad (37)$$

The summation is taken in score order, with M_0 being the highest score.

$$R = \frac{\text{your time}}{\text{fastest time by someone else}} \quad (38)$$

$$P(x) = e^{-1/x} x^{-d} \quad (39)$$

$$S(x) = \int_0^x P(y) dy \quad (40)$$

$$L(x) = S(x)/S(\infty) = \Gamma_n(d-1, 1/x) \quad (41)$$

Γ_n is the normalized upper incomplete gamma function.

$$H_y = I_y^g \quad (42)$$

$$N(x) = \sum_{i \neq y} \left[H_i P(k H_i x) \prod_{h \neq y, h \neq i} (1 - L(k H_h x)) \right] \quad (43)$$

$$Q(x) = P\left(b\left(\frac{1}{x} - 1\right)\right) N\left(\frac{1}{xR} - 1\right) \quad (44)$$

$$M = \text{individual map time score} = \left(\frac{\int_{x=0}^1 Q(x) \sqrt{x/(1-x)} dx}{\int_{x=0}^1 Q(x) dx} \right)^2 \quad (45)$$

$$F = e^{-M} + c \sum_{i < y} e^{-I_i} \quad (46)$$

$$E = \text{placement-corrected map score} = -\ln F / (1 - F) \quad (47)$$

M depends on I_i , whose initial condition is $I_i = 1$. If there's only one player on a map, then $M = 0$. You should use your library's integration function for M .

$0 < g < 1$: this is a tool for forcing convergence, taking in a person's I_i and deciding that their distribution should look closer to the default distribution than I_i says. 1 is more faithful. $g < 1$ because people who don't play much are given low scores, but their distribution is not as bad as it would seem.

$b > 0$: given an unknown racer, this says what his expected time is. Higher b means that the unknown racer is more likely to set really good times. Using $P(x)$ as our model is probably wrong here, but I haven't checked other models.

$k > 0$: given a specific racer and his I_i , this says what his expected time is. Higher k means that the racer is more likely to set really good times relative to his I_i . b and k scale together, so multiplying both of them by the same amount will multiply all time scores by a constant, which changes nothing.

However, higher time scores will result in less placement penalty for being defeated by a skilled runner. If b/k changes, I don't know what the effect will be.

$d > 1$: determines how likely it is for a runner to set a really bad time, no matter his skill level. Higher d means less likely to have terrible times.

$c > 0$: How much penalty to give for placement defeats. Higher c raises the penalty.

Each map has an optimal time o that nobody can go under. Someone who achieves time t gets score $o/(t-o)$. For example, if the optimum time is 10 seconds, then 11 seconds receives score 10, 10.01 seconds receives score 1000, and 10 seconds receives score ∞ . However, we don't know the optimal time.

Our tools for calculating your score are I_i , the estimated skill of each player on the map, and R , the ratio between your time and the best time by someone else. Using only a single time for R makes the model easier to understand, and more resistant to changes in the poorer times, which are more dependent on effort than skill. We use others' times for R , because you are compared to your opponents, not yourself. I_i means that a player doesn't need to worry that his poorer times will affect his identity.

The "competitiveness" input to the formula is I_i . If player A crushes player B on a few maps, and B crushes A on a few maps, then both players will be rated as better than the other. A 's score is better than B 's score, and B 's score is better than A 's score, and after a few iterations, A 's score will be infinity. So we keep in mind that I_i is not a good measure of runners' scores, only their best scores. It doesn't capture the distribution of scores either.

A player's time on a map has a long tail: getting a good time is rare, but getting a really bad time is common. This is because setting a good time takes grinding, but leaving bad times is easy.

Our model for player times is the distribution $e^{-1/x}x^{-1.1}$. This has two properties: fast decay at $x = 0$, and slow decay at $x = \infty$. The decay at infinity must be super-linear for the distribution to have finite integral, and $x^{-1.1}$ satisfies this.

The chance for a player to achieve (optimal time) $\times(1+x)$ is modeled as $H_i P(H_i x)$. The decay behavior means that close-to-optimal times are really rare, but really bad times are common. H_i is a sublinear power of I_i because I_i is an upper bound on performance, and penalizes those who have played few maps, sending their scores to zero. The sublinear growth is necessary for convergence, by drawing their distributions back to the mean.

W is the fastest time on a map, excepting your own. The chance that $W = (\text{optimal time}) \times (1+x)$ is $N(x)$, the first order statistic.

We'll model your expected times by $P(x)$. $Q(x)$ is the chance that the optimal time is $x \times (\text{your time})$. The score formula is (optimal time)/(your time - optimal time), so your estimated score is:

$$\text{expected individual map score} = \frac{\int_{x=0}^1 Q(x) (x/(1-x)) dx}{\int_{x=0}^1 Q(x) dx} \quad (48)$$

An uncertain score should be penalized over a certain score with the same expectation. This is because we use the top scores to calculate I_i . Our end result is:

$$M = \left(\frac{\int_{x=0}^1 Q(x) \sqrt{x/(1-x)} dx}{\int_{x=0}^1 Q(x) dx} \right)^2 \quad (49)$$

Note on implementation: the formula for $N(x)$ is $O(n^2)$. This can be improved to $O(n)$ by moving the product outside, but then you have to be careful about dividing by zero.

Our distributions are all single-parameter. They take in $H_i = I_c^{0.9}$.

Distributions should have long tails. Any player can set a bad time easily, and they often do so. If you set a really good time relative to someone, and that person has a short tail, then you receive a good score.

We tried $P(x) = x^{-\ln x}$ before. This is a log-normal distribution which was tested to have too short a tail (strong reltimes on low-population maps produce too high of a score). Its growth at 0 is the right rate.

The current distribution is $e^{-1/x} x^{-d}$. The $e^{-1/x}$ creates a strong 0 at $x = 0$, and the x^{-d} creates a long tail. Its dies at 0 too quickly. Higher d results in shorter tails, which means that runners are expected to set fewer bad times.

Trying to build something with $x^{-\ln x}$ growth at 0 and x^{-d} decay at infinity is a nice idea, but we also need our function to be quickly integrable. We use the CDF to calculate the first order statistic, which restricts the distributions we're allowed to use.

Placements: If you're defeated by someone, then consider their I_i , and decrease your own score.

Idea: $-\ln(e^{-M} + c \sum_i e^{-I_i})$. This isn't quite right (we want to add distributions again), but it's stateless, which means it's fast to calculate. Getting dominated by many people of high skill doesn't push your score down much. Compared to harmonic sums, the decrease is linear rather than multiplicative.

To avoid negative scores, we apply this transformation at the end: $x \rightarrow x/(1 + e^{-x})$. This grows linearly past 0, and shrinks to 0 for negative values. This formula gives us F and E .

Player aggregate score formula: The main requirement of aggregate weighting is that playing maps will never decrease your score. You should never feel restricted from trying new maps, and setting a garbage time is always better than setting no time. As a consequence, only your best maps count, because your overall score can't be any worse than if we only consider your best maps.

$$\sum_i 2^{-i/m} x_i \quad (50)$$

m is a constant. The sum iterates over the player's map scores x_i from highest to lowest. Once a player plays m maps, that player has filled out 50% of his score. Playing $2m$ maps fills out 75% of his score.

Setting m to a higher value incentivizes people to play more maps. Scoring systems aren't just about deciding who is the best, but also about making them do things. You could automatically increase m as more records are set, but that is for a later discussion.

7 TODO

1. a is not really acceleration, more like additional speed (check units)
2. $AT \leq 1$ and $\iota cT \leq 1$
3. OBS, zwob, multiple, sticky
4. wallbugs
5. frame rate dependency/impact
6. playbox hitbox sizes/origin location
7. skimtime: not enough -z ups
8. wallstrafe (2 keys better up to a certain point)
9. plasma climbing (i.e. optimal angle 70-73 (https://github.com/id-Software/Quake-III-Arena/blob/master/code/game/g_weapon.c#L140 SnapVectorTowards))
10. ceiling bugs (https://github.com/id-Software/Quake-III-Arena/blob/master/code/game/g_combat.c#L1189)
11. step-up

A Definitions

Vector

[Wik20a]

An n -dimensional vector is represented as

$$\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} = \|\mathbf{x}\| \hat{\mathbf{x}}, \quad \forall i \in \{1, 2, \dots, n\}, x_i \in \mathbb{R},$$

where $x_1, x_2, \dots, x_n$ are the components of $\mathbf{x}$. Alternatively, the vector is uniquely defined by its magnitude and direction. The magnitude is described by the *norm*¹³

$$\|\mathbf{x}\| = \sqrt{\mathbf{x}^T \mathbf{x}} = \sqrt{\sum_{i=1}^n x_i^2} = x,$$

and its direction with the *unit vector*

$$\hat{\mathbf{x}} = \frac{\mathbf{x}}{\|\mathbf{x}\|}, \quad \|\hat{\mathbf{x}}\| = 1.$$

The *dot product* of two vectors $\mathbf{a} = (a_1 \ a_2 \ \dots \ a_n)^T$ and $\mathbf{b} = (b_1 \ b_2 \ \dots \ b_n)^T$ can be defined both algebraically

$$\mathbf{a} \cdot \mathbf{b} = \mathbf{a}^T \mathbf{b} = \sum_{i=1}^n a_i b_i,$$

and geometrically

$$\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\| \|\mathbf{b}\| \cos \theta,$$

with θ the angle between the vectors. Note that when $\mathbf{a}$ and $\mathbf{b}$ are orthogonal, i.e. $\theta = \pi/2$, the dot product simplifies to

$$\mathbf{a} \cdot \mathbf{b} = 0.$$

Normal

[Wik20b, Wik20c, Wik20d]

A hyperplane, which is a flat $(n - 1)$ -dimensional subset of the n -dimensional Euclidean space, can be described by $n - 1$ linearly independent in-plane vectors $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_{n-1}$ together with a point on this hyperplane $\mathbf{x}_0$. Hence, its parameterization

$$\mathbf{r}(\mathbf{t}) = \mathbf{x}_0 + \mathbf{X} \mathbf{t},$$

with $\mathbf{X} = (\mathbf{x}_1 \ \mathbf{x}_2 \ \dots \ \mathbf{x}_{n-1})$ and $\mathbf{t} = (t_1 \ t_2 \ \dots \ t_{n-1})^T$. Any vector in the kernel of $\mathbf{X}$, i.e. $\mathbf{X} \mathbf{n} = \mathbf{0}$, is perpendicular to the hyperplane and is called a *normal*.

Projection

[Wik21]

An orthogonal projection can be performed by a matrix multiplication. To project *onto* the vector $\mathbf{x}$, the matrix is defined by the outer product

$$\mathbf{P}_x = \hat{\mathbf{x}} \hat{\mathbf{x}}^T = \mathbf{P}_x^T = \mathbf{P}_x^k, \quad k \in \mathbb{N}^+.$$

Similarly, to project *along* the vector $\mathbf{x}$ onto its corresponding hyperplane, the matrix is defined by

$$\mathbf{M}_x = \mathbf{I} - \mathbf{P}_x = \mathbf{M}_x^T = \mathbf{M}_x^k, \quad k \in \mathbb{N}^+,$$

with $\mathbf{I}$ the identity matrix.

¹³The norm $\|\cdot\|$ represents the Euclidean norm, i.e. ℓ^2 -norm, unless specified otherwise.

B Code snippets

B.1 AngleVectors

```
1238 void AngleVectors( const vec3_t angles, vec3_t forward, vec3_t right, vec3_t up) {
1239     float          angle;
1240     static float    sr, sp, sy, cr, cp, cy;
1241     // static to help MS compiler fp bugs
1242
1243     angle = angles[YAW] * (M_PI*2 / 360);
1244     sy = sin(angle);
1245     cy = cos(angle);
1246     angle = angles[PITCH] * (M_PI*2 / 360);
1247     sp = sin(angle);
1248     cp = cos(angle);
1249     angle = angles[ROLL] * (M_PI*2 / 360);
1250     sr = sin(angle);
1251     cr = cos(angle);
1252
1253     if (forward)
1254     {
1255         forward[0] = cp*cy;
1256         forward[1] = cp*sy;
1257         forward[2] = -sp;
1258     }
1259     if (right)
1260     {
1261         right[0] = (-1*sr*sp*cy+-1*cr*-sy);
1262         right[1] = (-1*sr*sp*sy+-1*cr*cy);
1263         right[2] = -1*sr*cp;
1264     }
1265     if (up)
1266     {
1267         up[0] = (cr*sp*cy+-sr*-sy);
1268         up[1] = (cr*sp*sy+-sr*cy);
1269         up[2] = cr*cp;
1270     }
1271 }
```

Listing 10: Code snippet from `code/game/q_math.c`.

B.2 PM_Friction

```
165 /*
166 =====
167 PM_Friction
168
169 Handles both ground friction and water friction
170 =====
171 */
172 static void PM_Friction( void ) {
173     vec3_t      vec;
174     float        *vel;
175     float        speed, newspeed, control;
176     float        drop;
177
178     vel = pm->ps->velocity;
179
180     VectorCopy( vel, vec );
181     if ( pml.walking ) {
182         vec[2] = 0;          // ignore slope movement
183     }
184
185     speed = VectorLength(vec);
186     if (speed < 1) {
187         vel[0] = 0;
188         vel[1] = 0;          // allow sinking underwater
189         //FIXME: still have z friction underwater?
190         return;
191     }
```

```

191     }
192
193     drop = 0;
194
195     // apply ground friction
196     if ( pm->waterlevel <= 1 ) {
197         if ( pml.walking && !(pml.groundTrace.surfaceFlags & SURF_SLICK) ) {
198             // if getting knocked back, no friction
199             if ( ! (pm->ps->pm_flags & PMF_TIME_KNOCKBACK) ) {
200                 control = speed < pm_stopspeed ? pm_stopspeed : speed;
201                 drop += control*pm_friction*pml.frameTime;
202             }
203         }
204     }
205
206     // apply water friction even if just wading
207     if ( pm->waterlevel ) {
208         drop += speed*pm_waterfriction*pm->waterlevel*pml.frameTime;
209     }
210
211     // apply flying friction
212     if ( pm->ps->powerups[PW_FLIGHT] ) {
213         drop += speed*pm_flightfriction*pml.frameTime;
214     }
215
216     if ( pm->ps->pm_type == PM_SPECTATOR ) {
217         drop += speed*pm_spectatorfriction*pml.frameTime;
218     }
219
220     // scale the velocity
221     newspeed = speed - drop;
222     if ( newspeed < 0 ) {
223         newspeed = 0;
224     }
225     newspeed /= speed;
226
227     vel[0] = vel[0] * newspeed;
228     vel[1] = vel[1] * newspeed;
229     vel[2] = vel[2] * newspeed;
230 }

```

Listing 11: Code snippet from `code/game/bg_pmove.c`.

B.3 PM_CmdScale

```

281  /*
282  =====
283  PM_CmdScale
284
285  Returns the scale factor to apply to cmd movements
286  This allows the clients to use axial -127 to 127 values for all directions
287  without getting a sqrt(2) distortion in speed.
288  =====
289  */
290  static float PM_CmdScale( usercmd_t *cmd ) {
291      int         max;
292      float       total;
293      float       scale;
294
295      max = abs( cmd->forwardmove );
296      if ( abs( cmd->rightmove ) > max ) {
297          max = abs( cmd->rightmove );
298      }
299      if ( abs( cmd->upmove ) > max ) {
300          max = abs( cmd->upmove );
301      }
302      if ( !max ) {
303          return 0;
304      }
305

```

```
306     total = sqrt( cmd->forwardmove * cmd->forwardmove
307                   + cmd->rightmove * cmd->rightmove + cmd->upmove * cmd->upmove );
308     scale = (float)pm->ps->speed * max / ( 127.0 * total );
309
310     return scale;
311 }
```

Listing 12: Code snippet from `code/game/bg_pmove.c`.

C Derivations

Common assumptions in these derivations

$$0 \leq AT \leq 1, \quad (51)$$

$$0 \leq \iota cT \leq 1, \quad (52)$$

hence the following also holds

$$0 \leq AT(2 - AT) \leq 1, \quad (53)$$

$$0 \leq \iota cT(2 - \iota cT) \leq 1. \quad (54)$$

C.1 $\delta_{\min}$

(6)

$$\begin{aligned} \|\underline{v}_f + (s - \underline{v}_f \cos \delta_{\min})\underline{\hat{a}}\|^2 &= \|\underline{v}\|^2, \\ (\underline{v}_f + (s - \underline{v}_f \cos \delta_{\min})\underline{\hat{a}})^T (\underline{v}_f + (s - \underline{v}_f \cos \delta_{\min})\underline{\hat{a}}) &= \underline{v}^T \underline{v}, \\ 2(s - \underline{v}_f \cos \delta_{\min})\underline{\hat{a}}^T \underline{v}_f + (s - \underline{v}_f \cos \delta_{\min})^2 \underline{\hat{a}}^T \underline{\hat{a}} &= \underline{v}^T \underline{v} - \underline{v}_f^T \underline{v}_f, \\ 2(s - \underline{v}_f \cos \delta_{\min})\underline{v}_f \cos \delta_{\min} + (s - \underline{v}_f \cos \delta_{\min})^2 &= \underline{v}^2 - \underline{v}_f^2, \\ s^2 - \underline{v}_f^2 \cos^2 \delta_{\min} &= \underline{v}^2 - \underline{v}_f^2, \\ \delta_{\min} &= \arccos \left(\frac{\sqrt{s^2 - \underline{v}^2 + \underline{v}_f^2}}{\underline{v}_f} \right). \end{aligned}$$

Alternatively, using equation (2) we find the extended form

$$\begin{aligned} \delta_{\min} &= \arccos \left(\frac{\sqrt{s^2 - \underline{v}^2 + \underline{v}_f^2}}{\underline{v}_f} \right) \\ &= \arccos \left(\frac{\sqrt{s^2 - \underline{v}^2 + (\underline{v} - \underline{v} \iota cT)^2}}{\underline{v} - \underline{v} \iota cT} \right) \\ &= \arccos \left(\sqrt{\frac{s^2 - \underline{v}^2 \iota cT(2 - \iota cT)}{\underline{v}^2 - \underline{v}^2 \iota cT(2 - \iota cT)}} \right) \end{aligned}$$

- Take square root of positive number + (54)

$$\begin{aligned} &\underline{v}^2 \iota cT(2 - \iota cT) \leq s^2, \\ &\begin{cases} \underline{v}^2 \leq s^2, & \underline{v} \leq dcT \\ dcT(2\underline{v} - dcT) \leq s^2, & dcT < \underline{v} \leq d \\ \underline{v}^2 cT(2 - cT) \leq s^2, & \underline{v} > d \end{cases} \\ &\begin{cases} \underline{v} \leq s, & \underline{v} \leq dcT \\ \underline{v} \leq \frac{s^2 + d^2 c^2 T^2}{2dcT}, & dcT < \underline{v} \leq d \\ \underline{v} \leq s \sqrt{\frac{1}{cT(2 - cT)}}, & \underline{v} > d \end{cases} \\ &\underline{v} \leq s \sqrt{\frac{1}{\iota cT(2 - \iota cT)}}. \end{aligned} \quad (55)$$

- Using equation (5), it's only valid in range

$$\begin{aligned} 0 &\leq s - \underline{v}_f \cos \delta_{\min} \leq \underline{a}, \\ s - \underline{a} &\leq \underline{v}_f \cos \delta_{\min} \leq s, \\ s - sAT &\leq \sqrt{s^2 - \underline{v}^2 + \underline{v}_f^2} \leq s, \end{aligned}$$

Under condition (55), then every term is positive, take square.

$$\begin{aligned} s^2 - s^2 AT(2 - AT) &\leq s^2 - \underline{v}^2 + \underline{v}_f^2 \leq s^2, \\ 0 &\leq \underline{v}^2 - \underline{v}_f^2 \leq s^2 AT(2 - AT), \\ 0 &\leq \underline{v}^2 \iota c T(2 - \iota c T) \leq s^2 AT(2 - AT), \end{aligned}$$

Since (53) and (54)

$$\begin{aligned} 0 &\leq \underline{v}^2 \leq s^2 \frac{AT(2 - AT)}{\iota c T(2 - \iota c T)}, \\ 0 &\leq \underline{v} \leq s \sqrt{\frac{AT(2 - AT)}{\iota c T(2 - \iota c T)}}, \end{aligned} \tag{56}$$

which is more strict than (55), because of (53). It even simplifies to (11)

$$0 \leq \underline{v} \leq \underline{v}_{\max}, \tag{57}$$

- The numerator smaller or equal than the denominator (acos range)

$$\begin{aligned} s^2 - \underline{v}^2 + \underline{v}_f^2 &\leq \underline{v}_f^2, \\ \underline{v} &\geq s \end{aligned}$$

The $\delta_{\min}$ valid interval/range

$$\begin{aligned} s &\leq \underline{v} \leq \underline{v}_{\max}, \\ \text{acos}\left(\frac{\sqrt{\underline{v}_f^2}}{\underline{v}_f}\right) &\leq \delta_{\min} \leq \text{acos}\left(\frac{\sqrt{s^2 - \underline{v}^2 \iota c T(2 - \iota c T)}}{\underline{v}(1 - \iota c T)}\right), \\ \text{acos}(1) &\leq \delta_{\min} \leq \text{acos}\left(\frac{\sqrt{s^2 - s^2 AT(2 - AT)}}{\underline{v}(1 - \iota c T)}\right), \\ 0 &\leq \delta_{\min} \leq \text{acos}\left(\frac{s(1 - AT)}{\underline{v}(1 - \iota c T)}\right), \\ 0 &\leq \delta_{\min} \leq \text{acos}\left(\frac{s(1 - AT)}{s \sqrt{\frac{AT(2 - AT)}{\iota c T(2 - \iota c T)}}(1 - \iota c T)}\right), \\ 0 &\leq \delta_{\min} \leq \text{acos}\left(\sqrt{\frac{\iota c T(2 - \iota c T)}{AT(2 - AT)}} \frac{(1 - AT)}{(1 - \iota c T)}\right), \\ 0 &\leq \delta_{\min} \leq \text{acos}\left(\sqrt{\frac{(1 - (1 - \iota c T)^2)(1 - AT)^2}{(1 - (1 - AT)^2)(1 - \iota c T)^2}}\right), \\ 0 &\leq \delta_{\min} \leq \delta_{\underline{v}_{\max}}, \end{aligned}$$

Note that it simplifies to δ_{opt} when approaching the maximum ground speed $\underline{v}_{\max}$.

C.2 δ_{opt}

(7)

$$\begin{aligned} \|\mathbf{P}_{\underline{a}} \underline{v}_f\| + \|\underline{a}\| &= s \|\hat{\underline{a}}\|, \\ \underline{v}_f \cos \delta_{\text{opt}} + \underline{a} &= s, \\ \delta_{\text{opt}} &= \text{acos}\left(\frac{s - \underline{a}}{\underline{v}_f}\right). \end{aligned}$$

Alternatively, using equation (2) we find the extended form

$$\begin{aligned}\delta_{\text{opt}} &= \text{acos} \left(\frac{s - \underline{a}}{\underline{v}_f} \right), \\ &= \text{acos} \left(\frac{s(1 - AT)}{\underline{v}(1 - \iota c T)} \right)\end{aligned}$$

- (5), it's only valid in range

$$\begin{aligned}\underline{v}_f \cos \delta_{\text{opt}} &\leq s - \underline{a}, \\ s - \underline{a} &\leq s - \underline{a},\end{aligned}$$

which is always valid.

- The numerator smaller or equal than the denominator (acos range) + (51) and (52)

$$\begin{aligned}s(1 - AT) &\leq \underline{v}(1 - \iota c T), \\ \underline{v} &\geq s \frac{(1 - AT)}{(1 - \iota c T)},\end{aligned}$$

The δ_{opt} valid interval/range

$$\begin{aligned}s \frac{(1 - AT)}{(1 - \iota c T)} &\leq \underline{v}, \\ \text{acos} \left(\frac{s(1 - AT)}{s(1 - AT)} \right) &\leq \delta_{\text{opt}}, \\ 0 &\leq \delta_{\text{opt}},\end{aligned}$$

C.3 δ_{max}

(8)

$$\begin{aligned}\|\underline{v}_f + \underline{a}\|^2 &= \|\underline{v}\|^2, \\ (\underline{v}_f + \underline{a})^T (\underline{v}_f + \underline{a}) &= \underline{v}^T \underline{v}, \\ 2\underline{a}^T \underline{v}_f + \underline{a}^T \underline{a} &= \underline{v}^T \underline{v} - \underline{v}_f^T \underline{v}_f, \\ 2\underline{a} \underline{v}_f \cos \delta_{\text{max}} + \underline{a}^2 &= \underline{v}^2 - \underline{v}_f^2, \\ \delta_{\text{max}} &= \text{acos} \left(\frac{\underline{v}^2 - \underline{v}_f^2 - \underline{a}^2}{2\underline{a} \underline{v}_f} \right).\end{aligned}$$

Alternatively, using equation (2) we find the extended form

$$\begin{aligned}\delta_{\text{max}} &= \text{acos} \left(\frac{\underline{v}^2 - \underline{v}_f^2 - \underline{a}^2}{2\underline{a} \underline{v}_f} \right), \\ &= \text{acos} \left(\frac{\underline{v}^2 \iota c T (2 - \iota c T) - s^2 A^2 T^2}{2s \underline{v} AT (1 - \iota c T)} \right), \\ &= \text{acos} \left(\frac{\underline{v} \underline{v} (\iota c T) (2 - \iota c T) - s s (AT) (AT)}{s \underline{v} (AT) (2 - \iota c T) - s \underline{v} (AT) (\iota c T)} \right),\end{aligned}$$

- (5), it's only valid in range + (53) and (54)

$$\begin{aligned}
\underline{v}_f \cos \delta_{\max} &\leq s - a, \\
\frac{\underline{v}^2 - \underline{v}_f^2 - \underline{a}^2}{2\underline{a}} &\leq s - a, \\
\underline{v}^2 - \underline{v}_f^2 &\leq 2as - a^2, \\
\underline{v}^2 \iota c T (2 - \iota c T) &\leq s^2 AT (2 - AT), \\
\underline{v} &\leq s \sqrt{\frac{AT(2 - AT)}{\iota c T (2 - \iota c T)}},
\end{aligned}$$

- The numerator can be both positive and negative. Numerator negative, in this case the numerator is smaller or equal than the denominator (acos range) when

$$\begin{aligned}
-\underline{v}^2 + \underline{v}_f^2 + \underline{a}^2 &\leq 2\underline{a}\underline{v}_f, \\
-\underline{v}^2 \iota c T (2 - \iota c T) + \underline{a}^2 &\leq 2\underline{a}\underline{v}(1 - \iota c T), \\
\underline{a}^2 + 2\underline{a}(\underline{v}\iota c T) + (\underline{v}\iota c T)^2 &\leq 2\underline{a}\underline{v} + 2\underline{v}^2 \iota c T, \\
(\underline{a} + \underline{v}\iota c T)^2 &\leq 2\underline{v}(\underline{a} + \underline{v}\iota c T),
\end{aligned}$$

since (52)

$$\begin{aligned}
\underline{a} + \underline{v}\iota c T &\leq 2\underline{v}, \\
\underline{v} &\geq s \frac{AT}{2 - \iota c T},
\end{aligned}$$

- Numerator positive, in this case the numerator is smaller or equal than the denominator (acos range) when

$$\begin{aligned}
\underline{v}^2 - \underline{v}_f^2 - \underline{a}^2 &\leq 2\underline{a}\underline{v}_f, \\
\underline{v}^2 \iota c T (2 - \iota c T) - \underline{a}^2 &\leq 2\underline{a}\underline{v}(1 - \iota c T), \\
-\underline{a}^2 + 2\underline{a}(\underline{v}\iota c T) - (\underline{v}\iota c T)^2 &\leq 2\underline{a}\underline{v} - 2\underline{v}^2 \iota c T, \\
-(\underline{a} - \underline{v}\iota c T)^2 &\leq 2\underline{v}(\underline{a} - \underline{v}\iota c T),
\end{aligned} \tag{58}$$

$\underline{a} - \underline{v}\iota c T$ can be both positive and negative. When $\underline{a} - \underline{v}\iota c T \leq 0$ (58) becomes with (52)

$$\begin{aligned}
-(\underline{a} - \underline{v}\iota c T) &\geq 2\underline{v}, \\
\underline{v} &\leq -s \frac{AT}{2 - \iota c T},
\end{aligned}$$

which is never true, because $\underline{v} \geq 0$. When $\underline{a} - \underline{v}\iota c T \geq 0$ (58) becomes with (52)

$$-(\underline{a} - \underline{v}\iota c T) \leq 2\underline{v},$$

which is always true. Hence,

$$\begin{aligned}
\underline{a} - \underline{v}\iota c T &\geq 0, \\
\underline{v} &\leq s \frac{AT}{\iota c T},
\end{aligned}$$

The $\delta_{\max}$ valid interval/range

$$\begin{aligned}
s \frac{AT}{2 - \iota c T} \leq \underline{v} &\leq \begin{cases} s \frac{AT}{\iota c T}, & AT \leq \iota c T, \\ s \sqrt{\frac{AT(2-AT)}{\iota c T(2-\iota c T)}}, & AT > \iota c T, \end{cases} \\
\text{acos} \left(\frac{\frac{s^2 A^2 T^2 \iota c T}{2 - \iota c T} - s^2 A^2 T^2}{s^2 A^2 T^2 - \frac{s^2 A^2 T^2 \iota c T}{2 - \iota c T}} \right) \geq \delta_{\max} &\geq \begin{cases} \text{acos} \left(\frac{\frac{s^2 A^2 T^2 (2 - \iota c T)}{\iota c T} - s^2 A^2 T^2}{\frac{s^2 A^2 T^2 (2 - \iota c T)}{\iota c T} - s^2 A^2 T^2} \right), & AT \leq \iota c T, \\ \text{acos} \left(\frac{s^2 AT(2-AT) - s^2 A^2 T^2}{2s \underline{v} AT(1 - \iota c T)} \right), & AT > \iota c T, \end{cases} \\
\text{acos}(-1) \geq \delta_{\max} &\geq \begin{cases} \text{acos}(1), & AT \leq \iota c T, \\ \text{acos} \left(\frac{2s s AT(1-AT)}{2s \underline{v} AT(1 - \iota c T)} \right), & AT > \iota c T, \end{cases} \\
\pi \geq \delta_{\max} &\geq \begin{cases} 0, & AT \leq \iota c T, \\ \text{acos} \left(\frac{s(1-AT)}{\underline{v}(1 - \iota c T)} \right), & AT > \iota c T, \end{cases}
\end{aligned}$$

Note that it simplifies to δ_{opt} when approaching the maximum ground speed $\underline{v}_{\max}$.

C.4 $\bar{\delta}_{\min}$

(12)

$$\begin{aligned}
\| \underline{\mathbf{P}}_{\underline{v}} (\underline{\mathbf{v}}_f + (s - \underline{v}_f \cos \bar{\delta}_{\min}) \underline{\hat{\mathbf{a}}}) \| &= \| \underline{\mathbf{v}} \|, \\
\| \underline{v}_f \underline{\hat{\mathbf{v}}} + (s - \underline{v}_f \cos \bar{\delta}_{\min}) \cos \bar{\delta}_{\min} \underline{\hat{\mathbf{v}}} \| &= \underline{v}, \\
(\underline{v}_f + s \cos \bar{\delta}_{\min} - \underline{v}_f \cos^2 \bar{\delta}_{\min}) \| \underline{\hat{\mathbf{v}}} \| &= \underline{v}, \\
s \cos \bar{\delta}_{\min} - \underline{v}_f \cos^2 \bar{\delta}_{\min} &= \underline{v} - \underline{v}_f, \quad D = s^2 - 4\underline{v}_f(\underline{v} - \underline{v}_f), \\
\bar{\delta}_{\min} &= \text{acos} \left(\frac{s \pm \sqrt{s^2 - 4\underline{v}_f(\underline{v} - \underline{v}_f)}}{2\underline{v}_f} \right).
\end{aligned}$$

Since $s \geq \underline{a}$ and the acceleration should not be greater than the full acceleration $\underline{a}$, mathematically

$$0 \leq s - \underline{v}_f \cos \bar{\delta}_{\min} = s/2 \mp \sqrt{s^2 - 4\underline{v}_f(\underline{v} - \underline{v}_f)} / 2 \leq \underline{a}.$$

We find that this only holds for

$$\bar{\delta}_{\min} = \text{acos} \left(\frac{s + \sqrt{s^2 - 4\underline{v}_f(\underline{v} - \underline{v}_f)}}{2\underline{v}_f} \right).$$

C.5 $\bar{\delta}_{\max}$

(13)

$$\begin{aligned}
\| \underline{\mathbf{P}}_{\underline{v}} (\underline{\mathbf{v}}_f + \underline{\mathbf{a}}) \| &= \| \underline{\mathbf{v}} \|, \\
\| \underline{v}_f \underline{\hat{\mathbf{v}}} + \underline{a} \cos \bar{\delta}_{\max} \underline{\hat{\mathbf{v}}} \| &= \underline{v}, \\
(\underline{v}_f + \underline{a} \cos \bar{\delta}_{\max}) \| \underline{\hat{\mathbf{v}}} \| &= \underline{v}, \\
\underline{a} \cos \bar{\delta}_{\max} &= \underline{v} - \underline{v}_f, \\
\bar{\delta}_{\max} &= \text{acos} \left(\frac{\underline{v} - \underline{v}_f}{\underline{a}} \right). \\
\bar{\delta}_{\max} &= \text{acos} \left(\frac{\underline{v} - \underline{v}_f}{\underline{a}} \right), \\
&= \text{acos} \left(\frac{\underline{v} \iota c T}{s AT} \right).
\end{aligned}$$

- (5), it's only valid in range

$$\begin{aligned}\underline{v}_f \cos \bar{\delta}_{\max} &\leq s - a, \\ \frac{\underline{v}^2 \iota c T (1 - \iota c T)}{s A T} &\leq s(1 - A T), \\ \underline{v}^2 \iota c T (1 - \iota c T) &\leq s^2 A T (1 - A T), \\ \underline{v} &\leq s \sqrt{\frac{A T (1 - A T)}{\iota c T (1 - \iota c T)}},\end{aligned}$$

- The numerator is smaller or equal than the denominator (acos range) when

$$\begin{aligned}\underline{v} \iota c T &\leq s A T, \\ \underline{v} &\leq s \frac{A T}{\iota c T},\end{aligned}$$

The $\bar{\delta}_{\max}$ valid interval/range

$$\begin{aligned}0 \leq \underline{v} &\leq \begin{cases} s \frac{A T}{\iota c T}, & A T \leq \iota c T, \\ s \sqrt{\frac{A T (1 - A T)}{\iota c T (1 - \iota c T)}}, & A T > \iota c T, \end{cases} \\ \text{acos}(0) \geq \bar{\delta}_{\max} &\geq \begin{cases} \text{acos}(1), & A T \leq \iota c T, \\ \text{acos}\left(\frac{\iota c T}{A T} \sqrt{\frac{A T (1 - A T)}{\iota c T (1 - \iota c T)}}\right), & A T > \iota c T, \end{cases} \\ \frac{\pi}{2} \geq \bar{\delta}_{\max} &\geq \begin{cases} 0, & A T \leq \iota c T, \\ \text{acos}\left(\sqrt{\frac{\iota c T (1 - A T)}{A T (1 - \iota c T)}}\right), & A T > \iota c T, \end{cases}\end{aligned}$$

C.6 $\underline{v}_{\max}$

We find the *maximum ground speed* (11) by

$$(8) = (6),$$

$$\frac{(\underline{v}^2 - \underline{v}_f^2) - \underline{a}^2}{2a\underline{v}_f} = \frac{\sqrt{s^2 - (\underline{v}^2 - \underline{v}_f^2)}}{\underline{v}_f},$$

$$\left((\underline{v}^2 - \underline{v}_f^2) - \underline{a}^2\right)^2 = 4\underline{a}^2(s^2 - (\underline{v}^2 - \underline{v}_f^2)),$$

$$(\underline{v}^2 - \underline{v}_f^2)^2 - 2\underline{a}^2(\underline{v}^2 - \underline{v}_f^2) + \underline{a}^4 = 4\underline{a}^2 s^2 - 4\underline{a}^2(\underline{v}^2 - \underline{v}_f^2),$$

$$(\underline{v}^2 - \underline{v}_f^2)^2 + 2\underline{a}^2(\underline{v}^2 - \underline{v}_f^2) + \underline{a}^4 = 4\underline{a}^2 s^2,$$

$$\left((\underline{v}^2 - \underline{v}_f^2) + \underline{a}^2\right)^2 = 4\underline{a}^2 s^2,$$

$$\underline{v}^2 - \underline{v}_f^2 = 2\underline{a}s - \underline{a}^2, \quad (59)$$

$$\underline{v}^2 \iota c T (2 - \iota c T) = s^2 A T (2 - A T), \quad (60)$$

$$\begin{cases} \underline{v}^2 = s^2 A T (2 - A T), & \underline{v} \leq d c T, \\ d c T (2\underline{v} - d c T) = s^2 A T (2 - A T), & d c T < \underline{v} \leq d, \\ \underline{v}^2 c T (2 - c T) = s^2 A T (2 - A T), & \underline{v} > d, \end{cases} \quad (61)$$

$$\underline{v} = \begin{cases} s \sqrt{A T (2 - A T)}, & \underline{v} \leq d c T, \\ \frac{s^2 A T (2 - A T) + d^2 c^2 T^2}{2 d c T}, & d c T < \underline{v} \leq d, \\ s \sqrt{\frac{A T (2 - A T)}{c T (2 - c T)}}, & \underline{v} > d \end{cases} \quad (62)$$

assuming that $\underline{v} \geq d$ and substituting $\underline{v}_f$ and $\underline{a}$ using equation (2) and (4) respectively, we get

$$\underline{v}^2 c T (2 - c T) = s^2 A T (2 - A T),$$

$$\underline{v}_{\max} = s \sqrt{\frac{A(2 - A T)}{c(2 - c T)}}.$$

As one might notice, the angle δ_{opt} also meets at this point, because

$$\begin{aligned} (8) &= (7), \\ \frac{(\underline{v}^2 - \underline{v}_f^2) - \underline{a}^2}{2\underline{a}\underline{v}_f} &= \frac{s - \underline{a}}{\underline{v}_f}, \\ \underline{v}^2 - \underline{v}_f^2 &= 2\underline{a}s - \underline{a}^2 \end{aligned}$$

is identical to equation (59).

C.6.1 $\underline{v}_{\text{max}}$ alternative

To solve for $\underline{v}_{\text{max}}$, equate (18) = $\underline{v}$.

$$\begin{aligned} \underline{v} &= \sqrt{((1 - \iota c T)\underline{v})^2 + 2s\underline{a} - \underline{a}^2} \\ \underline{v}^2 &= \underline{v}^2 (1^2 + (\iota c T)^2 - 2\iota c T) + s^2(2AT - A^2 T^2) \\ \underline{v}^2(\iota c T(\iota c T - 2)) &= -s^2(AT(2 - AT)) \\ \underline{v}^2 \iota c T(2 - \iota c T) &= s^2 AT(2 - AT), \end{aligned}$$

which is identical to (60).

Assuming $\underline{v} \geq d$, this becomes the same as above, (11).

C.7 $x_{z,k}$

To derive $x_{z,k}$ in terms of $x_{z,0}$ and $\nu_{z,0}$,

$$\begin{aligned}
x_{z,1} &= x_{z,0} + \left(\nu_{z,0} - \frac{gT_p}{2} \right) T_p \\
x_{z,2} &= x_{z,1} + \left(\nu_{z,1} - \frac{gT_p}{2} \right) T_p \\
&= x_{z,0} + \left(\nu_{z,0} - \frac{gT_p}{2} \right) T_p + \left(\nu_{z,0} - \lfloor gT_p \rfloor - \frac{gT_p}{2} \right) T_p \\
&= x_{z,0} + 2 \left(\nu_{z,0} - \frac{gT_p}{2} \right) T_p - \lfloor gT_p \rfloor T_p \\
x_{z,3} &= x_{z,1} + 2 \left(\nu_{z,1} - \frac{gT_p}{2} \right) T_p - \lfloor gT_p \rfloor T_p \\
&= x_{z,0} + \left(\nu_{z,0} - \frac{gT_p}{2} \right) T_p + 2 \left(\nu_{z,0} - \lfloor gT_p \rfloor - \frac{gT_p}{2} \right) T_p - \lfloor gT_p \rfloor T_p \\
&= x_{z,0} + 3 \left(\nu_{z,0} - \frac{gT_p}{2} \right) T_p - 3 \lfloor gT_p \rfloor T_p \\
x_{z,4} &= x_{z,1} + 3 \left(\nu_{z,1} - \frac{gT_p}{2} \right) T_p - 3 \lfloor gT_p \rfloor T_p \\
&= x_{z,0} + \left(\nu_{z,0} - \frac{gT_p}{2} \right) T_p + 3 \left(\nu_{z,0} - \lfloor gT_p \rfloor - \frac{gT_p}{2} \right) T_p - 3 \lfloor gT_p \rfloor T_p \\
&= x_{z,0} + 4 \left(\nu_{z,0} - \frac{gT_p}{2} \right) T_p - 6 \lfloor gT_p \rfloor T_p \\
x_{z,5} &= x_{z,1} + 4 \left(\nu_{z,1} - \frac{gT_p}{2} \right) T_p - 6 \lfloor gT_p \rfloor T_p \\
&= x_{z,0} + \left(\nu_{z,0} - \frac{gT_p}{2} \right) T_p + 4 \left(\nu_{z,0} - \lfloor gT_p \rfloor - \frac{gT_p}{2} \right) T_p - 6 \lfloor gT_p \rfloor T_p \\
&= x_{z,0} + 5 \left(\nu_{z,0} - \frac{gT_p}{2} \right) T_p - 10 \lfloor gT_p \rfloor T_p \\
x_{z,k} &= x_{z,0} + k \left(\nu_{z,0} - \frac{gT_p}{2} \right) T_p - \frac{k(k-1)}{2} \lfloor gT_p \rfloor T_p \\
&= x_{z,0} + k \left(\nu_{z,0} - \frac{gT_p}{2} \right) T_p + \frac{k}{2} \lfloor gT_p \rfloor T_p - \frac{k^2}{2} \lfloor gT_p \rfloor T_p \\
&= x_{z,0} + k \left(\nu_{z,0} - \frac{gT_p}{2} + \frac{\lfloor gT_p \rfloor}{2} \right) T_p - \frac{k^2}{2} \lfloor gT_p \rfloor T_p \\
&= x_{z,0} + k \left(\nu_{z,0} + \frac{\lfloor gT_p \rfloor - gT_p}{2} \right) T_p - \frac{k^2}{2} \lfloor gT_p \rfloor T_p,
\end{aligned}$$

deriving (25).

D Field of view

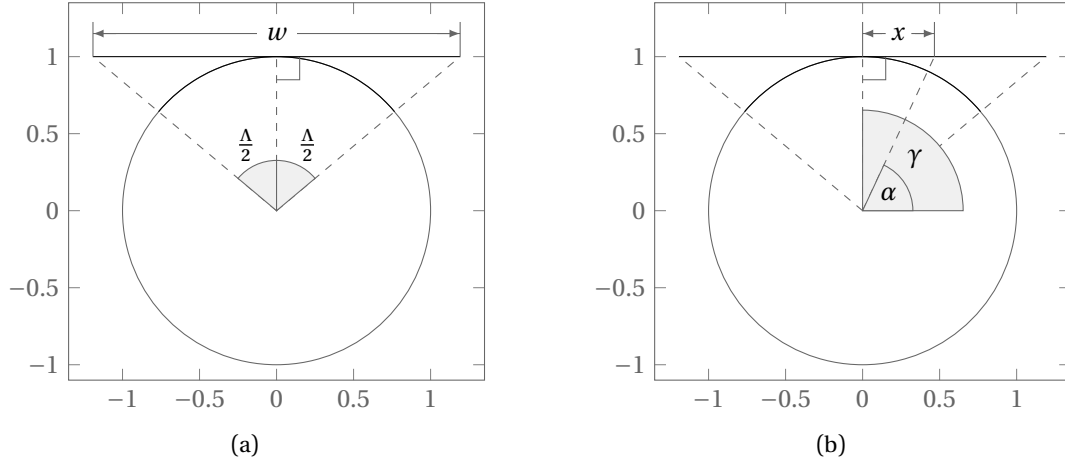


Figure 8: TODO

Field of view Λ_x^{14} (cg_fov)

$$w = 2 \tan \frac{\Lambda_x}{2}$$

$$S = \frac{W}{w} = \frac{W}{2 \tan \frac{\Lambda_x}{2}}$$

with $W = 640$ the width of the default screen SCREEN_WIDTH.

α is the desired yaw angle and γ current viewing direction

$$x = \begin{cases} -\tan \frac{\Lambda_x}{2}, & \gamma - \alpha \leq -\frac{\Lambda_x}{2}, \\ \tan(\gamma - \alpha), & |\gamma - \alpha| < \frac{\Lambda_x}{2}, \\ \tan \frac{\Lambda_x}{2}, & \gamma - \alpha \geq \frac{\Lambda_x}{2}, \end{cases}$$

$$X = \frac{W}{2} + \frac{W}{2} \frac{x}{\tan \frac{\Lambda_x}{2}},$$

$$X = \begin{cases} 0, & \gamma - \alpha \leq -\frac{\Lambda_x}{2}, \\ \frac{W}{2} + \frac{W}{2} \frac{\tan(\gamma - \alpha)}{\tan \frac{\Lambda_x}{2}}, & |\gamma - \alpha| < \frac{\Lambda_x}{2}, \\ W, & \gamma - \alpha \geq \frac{\Lambda_x}{2}, \end{cases}$$

$$X_2 - X_1 = -\frac{\tan(\alpha_{<} - \gamma)}{\tan\left(\frac{\text{cg_fov}}{2}\right)} \frac{\text{SCREEN_WIDTH}}{2} + \frac{\tan(\alpha_{>} - \gamma)}{\tan\left(\frac{\text{cg_fov}}{2}\right)} \frac{\text{SCREEN_WIDTH}}{2}$$

$$\text{width} = \frac{\text{SCREEN_WIDTH}}{2} \frac{\tan(\alpha_{>} - \gamma) - \tan(\alpha_{<} - \gamma)}{\tan\left(\frac{\text{cg_fov}}{2}\right)}$$

$$y = \tan(\beta - \rho)$$

$$Y = \frac{H}{2} + Sy$$

$$y = 240 + 320 \frac{\tan(\beta - \rho)}{\tan \frac{\Lambda}{2}}$$

¹⁴The field of view Λ_x is expressed in radians, while cg_fov in degrees.

References

- [Cho19] Chong JiangWei. Half-life physics reference. <https://www.jwchong.com/hl/>, 2019.
- [Chr19] Chris Welshman (inix). Quake articles. <https://www.funender.com/quake/articles/>, 2019.
- [ite21] items and +morosophos. RR ranking system. https://docs.google.com/document/d/1xm0Cu5lZmYAnxPPPk3g2Zb1pT_DuQg_7uvu2famXp2U/edit, 2021.
- [Wik20a] Wikipedia, The free encyclopedia. Dot product. https://en.wikipedia.org/wiki/Dot_product, 2020.
- [Wik20b] Wikipedia, The free encyclopedia. Euclidean space. https://en.wikipedia.org/wiki/Euclidean_space, 2020.
- [Wik20c] Wikipedia, The free encyclopedia. Hyperplane. <https://en.wikipedia.org/wiki/Hyperplane>, 2020.
- [Wik20d] Wikipedia, The free encyclopedia. Normal (geometry). [https://en.wikipedia.org/wiki/Normal_\(geometry\)](https://en.wikipedia.org/wiki/Normal_(geometry)), 2020.
- [Wik21] Wikipedia, The free encyclopedia. Projection matrix. https://en.wikipedia.org/wiki/Projection_matrix, 2021.